

You said:

Arquitectura 2.docx

Document

Casos de uso Final.docx

Document

Voy a desarrollar una aplicación de gestión de reservas de una biblioteca. Los casos de uso que tengo que desarrollar están en el documento de casos de uso, y la arquitectura está en el documento de arquitectura. Ahora mismo tengo descargado el inicializador de Spring boot. Voy a empezar con el mínimo producto como se indica en la arquitectura. Para el API voy a usar Spring Boot 4.0.1 con Java 25. He pensado hacer arquitectura MVC.

ChatGPT said:

Thought for 4 seconds

Perfecto. Con lo que dices (Spring Boot 4.0.1 + Java 25, empezar por el "mínimo producto" y una persona sola), **MVC te encaja como estilo de presentación**, pero para un API te conviene pensar en **arquitectura por capas** (Controller → Service → Repository) y **dominio claro** (entidades/reglas). Eso te mantiene el proyecto ordenado y escalable sin complicarte.

A partir de tus documentos:

- La **arquitectura mínima** propuesta (Beanstalk + RDS + Cognito + CloudWatch + Secrets) busca **reducir servicios y complejidad** para que sea realizable en ~2 meses por una sola persona.

Arquitectura 2

- Los **casos de uso** del MVP giran en torno a: catálogo, detalle, reservas, "mi cuenta" (préstamos/reservas/sanciones) y, para bibliotecario, préstamo/devolución/sanciones/ejemplares/configuración.

Casos de uso Final

Abajo te dejo una forma muy concreta de arrancar el MVP sin atascarte.

1) Qué haría como "mínimo producto" (backend API)

Si sigues literalmente el documento de casos de uso, el núcleo funcional mínimo razonable para tener “algo usable” sería:

Usuario autenticado

- **CU02** Consultar catálogo (búsqueda/listado)

Casos de uso Final

- **CU03** Ver detalle de un ejemplar

Casos de uso Final

- **CU04** Reservar un ejemplar

Casos de uso Final

- **CU05** Cancelar reserva

Casos de uso Final

- **CU06** Ver mis préstamos/reservas/sanciones

Casos de uso Final

- **CU09** Renovar préstamo

Casos de uso Final

Bibliotecario

- **CU07** Registrar préstamo

Casos de uso Final

- **CU08** Registrar devolución y generar sanción

Casos de uso Final

- (Opcional en MVP) **CU11** Gestión básica de ejemplares

Casos de uso Final

Dejaría para una segunda iteración:

- CU10 (gestión avanzada de sanciones), CU12 (envío a despacho profesor) y CU13 (configuración completa) salvo que tu arquitectura te exija sí o sí esos parámetros desde el inicio.

Casos de uso Final

2) Modelo de dominio mínimo (entidades que casi seguro necesitas)

Para cubrir esos CU sin inventarte cosas raras:

- **Usuario** (id externo del SSO/JWT "sub", tipo/rol)
- **Obra** (título, autor, materia... lo de catálogo)
- **Ejemplar** (id, obra, ubicación, estado: DISPONIBLE/PRESTADO/RESERVADO, referencia:boolean)
- **Reserva** (usuario, ejemplar, estado, fecha creación, posición cola si aplica)
- **Prestamo** (usuario, ejemplar, inicio, fin previsto, renovaciones, cerrado:boolean)
- **Sancion** (usuario, inicio, fin, causa, activa:boolean)

Eso ya te permite aplicar casi todas las reglas que aparecen en CU04, CU07, CU08, CU09.

Casos de uso Final

3) Estructura recomendada del proyecto (Spring "limpio" sin sobre-arquitectura)

Organiza así (simple y mantenible):

- `controller` (REST controllers)
- `service` (casos de uso / reglas de negocio)
- `repository` (Spring Data JPA)
- `domain` (entidades + enums)
- `dto` (request/response)

- security (JWT, roles)
- exception (errores controlados)

Esto es "MVC" en el sentido de **C (controller) + lógica (service) + modelo (domain)**, pero sin mezclar reglas en controladores.

4) Endpoints REST mínimos (mapeo 1:1 con casos de uso)

Catálogo

- GET /api/catalogo?query=... → CU02

Casos de uso Final

- GET /api/ejemplares/{id} → CU03

Casos de uso Final

Reservas

- POST /api/reservas (body: ejemplarId) → CU04

Casos de uso Final

- DELETE /api/reservas/{id} → CU05

Casos de uso Final

Mi cuenta

- GET /api/mi-cuenta → CU06

Casos de uso Final

- POST /api/prestamos/{id}/renovar → CU09

Casos de uso Final

Bibliotecario

- POST /api/admin/prestamos (usuarioid + ejemplarId) → CU07

Casos de uso Final

- `POST /api/admin/devoluciones` (ejemplarId) → CU08

Casos de uso Final

5) Seguridad (SSO/JWT) sin complicarte

Según tu arquitectura, la idea es SSO institucional → Cognito (o proveedor) → tu API valida JWT.

Arquitectura 2

En Spring esto suele ser:

- `spring-boot-starter-security`
- `spring-boot-starter-oauth2-resource-server`

Y configuras `issuer-uri` para que Spring valide firma/expiración/issuer automáticamente.

Roles: usa un claim tipo `cognito:groups` o similar, y mapea a `ROLE_BIBLIOTECARIO`, etc. Con eso proteges:

- `/api/admin/**` solo bibliotecario
 - el resto autenticado
-

6) Orden de implementación (para avanzar sin bloqueo)

1. **Levanta dominio + JPA + migraciones** (aunque sea `ddl-auto=update` al principio)
2. Implementa **CU02/CU03** (consulta es lo más rápido y te valida BD)

Casos de uso Final

3. Implementa **CU04/CU05** (reservas + reglas: sanción activa, referencia, colas)

Casos de uso Final

4. Implementa **CU07/CU08** (préstamo/devolución + sanción por retraso)

Casos de uso Final

5. Implementa **CU06/CU09** (mi cuenta + renovar con validaciones)

Casos de uso Final

6. Integra **JWT real** (si aún estabas mockeando) y cierra autorización por rol

Arquitectura 2

7) Nota rápida sobre “Spring Boot 4.0.1 + Java 25”

No te bloqueo la decisión (si ya la tienes). Solo te marco un riesgo práctico: **muchas guías/ejemplos y dependencias “de manual” estarán más testados en 3.x + Java 21/23**. Si vas con 4.0.1/25, perfecto, pero sé estricto con:

- versiones de dependencias
- compatibilidad de librerías (security, JPA, drivers)

Si quieres, en el siguiente paso te puedo dejar **un esqueleto de proyecto** (paquetes + clases vacías + DTOs + enums + primeros endpoints) alineado con estos CU, para que literalmente pegues y empieces a completar lógica.

Sources

You said:

si

ChatGPT said:

Genial—te dejo un **esqueleto listo para arrancar** (API REST por capas: Controller → Service → Repository), alineado con el MVP: **catálogo, detalle, reservas, mi cuenta, préstamo/devolución (admin)**.

Nota: lo pongo en **Gradle Kotlin DSL**. Si usas Maven, dímelo y te lo convierto.

1) Estructura de paquetes

```
src/main/java/com/tuorg/biblioteca/  
    BibliotecaApplication.java
```

```
    config/  
        JacksonConfig.java
```

```
    security/  
        SecurityConfig.java  
        JwtRoleConverter.java
```

```
    domain/  
        Usuario.java  
        Rol.java  
        Obra.java  
        Ejemplar.java  
        EstadoEjemplar.java  
        Reserva.java  
        EstadoReserva.java  
        Prestamo.java  
        Sancion.java
```

```
    dto/  
        catalogo/  
            ObraResumenDto.java  
            EjemplarDetalleDto.java  
        reservas/  
            ReservaCreateRequest.java  
            ReservaDto.java  
        micuenta/  
            MiCuentaDto.java  
            PrestamoDto.java  
            SancionDto.java
```

```
    ReservaDtoSimple.java
admin/
    CrearPrestamoRequest.java
    RegistrarDevolucionRequest.java
    PrestamoAdminDto.java

repository/
    UsuarioRepository.java
    ObraRepository.java
    EjemplarRepository.java
    ReservaRepository.java
    PrestamoRepository.java
    SancionRepository.java

service/
    CatalogoService.java
    ReservaService.java
    PrestamoService.java
    MiCuentaService.java

controller/
    CatalogoController.java
    ReservaController.java
    MiCuentaController.java
admin/
    PrestamoAdminController.java

exception/
    ApiError.java
    GlobalExceptionHandler.java
    NotFoundException.java
    BusinessException.java
```

2) build.gradle.kts (Spring Boot 4 + Java 25)

```
plugins {
    id("java")
    id("org.springframework.boot") version "4.0.1"
    id("io.spring.dependency-management") version "1.1.6"
}

group = "com.tuorg"
```



```

version = "0.0.1-SNAPSHOT"

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(25)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-validation")

    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    runtimeOnly("org.postgresql:postgresql")

    implementation("org.springframework.boot:spring-boot-starter-security")
    implementation("org.springframework.boot:spring-boot-starter-oauth2-resource-server")

    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.springframework.security:spring-security-test")
}

tasks.withType<Test> {
    useJUnitPlatform()
}

```

3) application.yml (local con Postgres + JWT)

```

server:
    port: 8080

spring:

```

```
datasource:
  url: jdbc:postgresql://localhost:5432/biblioteca
  username: biblioteca
  password: biblioteca
jpa:
  hibernate:
    ddl-auto: update
  properties:
    hibernate:
      format_sql: true
security:
  oauth2:
    resourceserver:
      jwt:
        issuer-uri: https://TU_ISSUER_AQUI
        # si usas jwk-set-uri en lugar de issuer:
        # jwk-set-uri: https://.../.well-known/jwks.json

logging:
  level:
    org.hibernate.SQL: debug
```

4) Dominio mínimo (JPA)

Enums

```
package com.tuorg.biblioteca.domain;

public enum Rol {
    USUARIO,
    BIBLIOTECARIO
}

package com.tuorg.biblioteca.domain;

public enum EstadoEjemplar {
    DISPONIBLE,
    PRESTADO,
    RESERVADO
}

package com.tuorg.biblioteca.domain;

public enum EstadoReserva {
```

```
    ACTIVA,  
    CANCELADA,  
    CUMPLIDA,  
    EXPIRADA  
}
```

Usuario (mapeado por subject del JWT)

```
package com.tuorg.biblioteca.domain;  
  
import jakarta.persistence.*;  
  
@Entity  
@Table(name = "usuarios")  
public class Usuario {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    // subject del token (Cognito/SSO)  
    @Column(nullable = false, unique = true)  
    private String externalId;  
  
    @Enumerated(EnumType.STRING)  
    @Column(nullable = false)  
    private Rol rol = Rol.USUARIO;  
  
    protected Usuario() {}  
  
    public Usuario(String externalId, Rol rol) {  
        this.externalId = externalId;  
        this.rol = rol;  
    }  
  
    public Long getId() { return id; }  
    public String getExternalId() { return externalId; }  
    public Rol getRol() { return rol; }  
}
```

Obra + Ejemplar

```
package com.tuorg.biblioteca.domain;  
  
import jakarta.persistence.*;  
  
@Entity  
@Table(name = "obras")
```

```

public class Obra {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String titulo;

    private String autor;
    private String materia;

    protected Obra() {}

    public Obra(String titulo, String autor, String materia) {
        this.titulo = titulo;
        this.autor = autor;
        this.materia = materia;
    }

    public Long getId() { return id; }
    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }
    public String getMateria() { return materia; }
}

package com.tuorg.biblioteca.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "ejemplares")
public class Ejemplar {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Obra obra;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoEjemplar estado = EstadoEjemplar.DISPONIBLE;

    @Column(nullable = false)
    private boolean referencia = false;
}

```

```

    private String ubicacion;

    protected Ejemplar() {}

    public Ejemplar(Obra obra, boolean referencia, String ubicacion) {
        this.obra = obra;
        this.referencia = referencia;
        this.ubicacion = ubicacion;
    }

    public Long getId() { return id; }
    public Obra getObra() { return obra; }
    public EstadoEjemplar getEstado() { return estado; }
    public boolean isReferencia() { return referencia; }
    public String getUbicacion() { return ubicacion; }

    public void setEstado(EstadoEjemplar estado) { this.estado = estado; }
}

```

Reserva, Préstamo, Sanción

```

package com.tuorg.biblioteca.domain;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "reservas")
public class Reserva {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Usuario usuario;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Ejemplar ejemplar;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoReserva estado = EstadoReserva.ACTIVA;

    @Column(nullable = false)

```

```

        private Instant creadaEn = Instant.now();

        protected Reserva() {}

        public Reserva(Usuario usuario, Ejemplar ejemplar) {
            this.usuario = usuario;
            this.ejemplar = ejemplar;
        }

        public Long getId() { return id; }
        public Usuario getUsuario() { return usuario; }
        public Ejemplar getEjemplar() { return ejemplar; }
        public EstadoReserva getEstado() { return estado; }
        public Instant getCreadaEn() { return creadaEn; }

        public void cancelar() { this.estado = EstadoReserva.CANCELADA; }
        public void cumplir() { this.estado = EstadoReserva.CUMPLIDA; }
    }
}

package com.tuorg.biblioteca.domain;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "prestamos")
public class Prestamo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Usuario usuario;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Ejemplar ejemplar;

    @Column(nullable = false)
    private LocalDate inicio;

    @Column(nullable = false)
    private LocalDate finPrevisto;

    @Column(nullable = false)
    private int renovaciones = 0;
}

```

```

@Column(nullable = false)
private boolean cerrado = false;

protected Prestamo() {}

public Prestamo(Usuario usuario, Ejemplar ejemplar, LocalDate inicio,
LocalDate finPrevisto) {
    this.usuario = usuario;
    this.ejemplar = ejemplar;
    this.inicio = inicio;
    this.finPrevisto = finPrevisto;
}

public Long getId() { return id; }
public Usuario getUsuario() { return usuario; }
public Ejemplar getEjemplar() { return ejemplar; }
public LocalDate getInicio() { return inicio; }
public LocalDate getFinPrevisto() { return finPrevisto; }
public int getRenovaciones() { return renovaciones; }
public boolean isCerrado() { return cerrado; }

public void renovar(LocalDate nuevoFin) {
    this.finPrevisto = nuevoFin;
    this.renovaciones++;
}

public void cerrar() { this.cerrado = true; }
}

package com.tuorg.biblioteca.domain;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "sanciones")
public class Sancion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private Usuario usuario;

    @Column(nullable = false)
    private LocalDate inicio;

```

```

@Column(nullable = false)
private LocalDate fin;

@Column(nullable = false)
private boolean activa = true;

private String causa;

protected Sancion() {}

public Sancion(Usuario usuario, LocalDate inicio, LocalDate fin,
String causa) {
    this.usuario = usuario;
    this.inicio = inicio;
    this.fin = fin;
    this.causa = causa;
    this.activa = true;
}

public Long getId() { return id; }
public Usuario getUsuario() { return usuario; }
public LocalDate getInicio() { return inicio; }
public LocalDate getFin() { return fin; }
public boolean isActiva() { return activa; }
public String getCausa() { return causa; }

public void desactivar() { this.activa = false; }
}

```

5) Repositorios (Spring Data JPA)

```

package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.Usuario;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByExternalId(String externalId);
}

```



```
package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.Obra;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface ObraRepository extends JpaRepository<Obra, Long>,
JpaSpecificationExecutor<Obra> {}

package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.Ejemplar;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EjemplarRepository extends JpaRepository<Ejemplar, Long>
{}

package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.EstadoReserva;
import com.tuorg.biblioteca.domain.Reserva;
import com.tuorg.biblioteca.domain.Usuario;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ReservaRepository extends JpaRepository<Reserva, Long> {
    List<Reserva> findByUsuarioIdAndEstado(Long usuarioId, EstadoReserva
estado);
    List<Reserva> findByUsuarioId(Long usuarioId);
}

package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.Prestamo;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PrestamoRepository extends JpaRepository<Prestamo, Long>
{
    List<Prestamo> findByUsuarioId(Long usuarioId);
}

package com.tuorg.biblioteca.repository;

import com.tuorg.biblioteca.domain.Sancion;
import java.time.LocalDate;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface SancionRepository extends JpaRepository<Sancion, Long> {
```

```
List<Sancion> findByUsuarioId(Long usuarioId);

boolean existsByUsuarioIdAndActivaIsTrueAndFinGreaterThanEqual(Long
usuarioId, LocalDate hoy);
}
```

6) Servicios (con reglas mínimas del MVP)

Excepciones

```
package com.tuorg.biblioteca.exception;

public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) { super(message); }
}

package com.tuorg.biblioteca.exception;

public class BusinessException extends RuntimeException {
    public BusinessException(String message) { super(message); }
}
```

CatalogoService (búsqueda simple)

```
package com.tuorg.biblioteca.service;

import com.tuorg.biblioteca.domain.Obra;
import com.tuorg.biblioteca.repository.ObraRepository;
import java.util.List;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;

@Service
public class CatalogoService {

    private final ObraRepository obraRepository;

    public CatalogoService(ObraRepository obraRepository) {
        this.obraRepository = obraRepository;
    }

    public List<Obra> buscar(String query) {
        if (query == null || query.isBlank()) {
            return obraRepository.findAll();
        }
    }
}
```

```

    }
    String q = "%" + query.trim().toLowerCase() + "%";

    Specification<Obra> spec = (root, cq, cb) ->
        cb.or(
            cb.like(cb.lower(root.get("titulo")), q),
            cb.like(cb.lower(root.get("autor")), q),
            cb.like(cb.lower(root.get("materia")), q)
        );

    return obraRepository.findAll(spec);
}
}

```

ReservaService (validación sanción + referencia + estado)

```

package com.tuorg.biblioteca.service;

import com.tuorg.biblioteca.domain.*;
import com.tuorg.biblioteca.exception.BusinessRuleException;
import com.tuorg.biblioteca.exception.NotFoundException;
import com.tuorg.biblioteca.repository.*;
import java.time.LocalDate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ReservaService {

    private final UsuarioRepository usuarioRepository;
    private final EjemplarRepository ejemplarRepository;
    private final ReservaRepository reservaRepository;
    private final SancionRepository sancionRepository;

    public ReservaService(
        UsuarioRepository usuarioRepository,
        EjemplarRepository ejemplarRepository,
        ReservaRepository reservaRepository,
        SancionRepository sancionRepository
    ) {
        this.usuarioRepository = usuarioRepository;
        this.ejemplarRepository = ejemplarRepository;
        this.reservaRepository = reservaRepository;
        this.sancionRepository = sancionRepository;
    }

    @Transactional

```

```

    public Reserva crearReserva(String externalUserId, Long ejemplarId) {
        Usuario usuario =
usuarioRepository.findByExternalId(externalUserId)
                .orElseThrow(() -> new NotFoundException("Usuario no
encontrado"));

        boolean sancionado =
sancionRepository.existsByUsuarioIdAndActivaIsTrueAndFinGreaterThanEqual(
                usuario.getId(), LocalDate.now()
        );
        if (sancionado) throw new BusinessException("No puedes
reservar: tienes una sanción activa.");

        Ejemplar ejemplar = ejemplarRepository.findById(ejemplarId)
                .orElseThrow(() -> new NotFoundException("Ejemplar no
encontrado"));

        if (ejemplar.isReferencia()) throw new BusinessException("No
se puede reservar un ejemplar de referencia.");
        if (ejemplar.getEstado() != EstadoEjemplar.DISPONIBLE) throw new
BusinessRuleException("Ejemplar no disponible.");

        Reserva reserva = new Reserva(usuario, ejemplar);
        ejemplar.setEstado(EstadoEjemplar.RESERVADO);

        // guardamos ambos (por cascada no está configurado)
        ejemplarRepository.save(ejemplar);
        return reservaRepository.save(reserva);
    }

    @Transactional
    public void cancelarReserva(String externalUserId, Long reservaId) {
        Usuario usuario =
usuarioRepository.findByExternalId(externalUserId)
                .orElseThrow(() -> new NotFoundException("Usuario no
encontrado"));

        Reserva reserva = reservaRepository.findById(reservaId)
                .orElseThrow(() -> new NotFoundException("Reserva no
encontrada"));

        if (!reserva.getUsuario().getId().equals(usuario.getId())) {
            throw new BusinessException("No puedes cancelar una
reserva de otro usuario.");
        }
        if (reserva.getEstado() != EstadoReserva.ACTIVA) {

```

```

        throw new BusinessException("Solo se pueden cancelar
reservas activas.");
    }

    reserva.cancelar();
    Ejemplar ejemplar = reserva.getEjemplar();
    ejemplar.setEstado(EstadoEjemplar.DISPONIBLE);

    reservaRepository.save(reserva);
    ejemplarRepository.save(ejemplar);
}
}

```

PrestamoService (admin: prestar/devolver) + renovar

```

package com.tuorg.biblioteca.service;

import com.tuorg.biblioteca.domain.*;
import com.tuorg.biblioteca.exception.BusinessRuleException;
import com.tuorg.biblioteca.exception.NotFoundException;
import com.tuorg.biblioteca.repository.*;
import java.time.LocalDate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class PrestamoService {

    private final UsuarioRepository usuarioRepository;
    private final EjemplarRepository ejemplarRepository;
    private final PrestamoRepository prestamoRepository;
    private final SancionRepository sancionRepository;

    // parámetros "MVP" (Luego lo llevas a tabla/config)
    private static final int DIAS_PRESTAMO = 14;
    private static final int DIAS_SANCION_POR_RETRASO = 7;
    private static final int MAX_RENOVACIONES = 1;

    public PrestamoService(
        UsuarioRepository usuarioRepository,
        EjemplarRepository ejemplarRepository,
        PrestamoRepository prestamoRepository,
        SancionRepository sancionRepository
    ) {
        this.usuarioRepository = usuarioRepository;
        this.ejemplarRepository = ejemplarRepository;
        this.prestamoRepository = prestamoRepository;
    }
}

```

```

        this.sancionRepository = sancionRepository;
    }

    @Transactional
    public Prestamo registrarPrestamo(Long usuarioId, Long ejemplarId) {
        Usuario usuario = usuarioRepository.findById(usuarioId)
            .orElseThrow(() -> new NotFoundException("Usuario no
encontrado"));

        boolean sancionado =
sancionRepository.existsByUsuarioIdAndActivaIsTrueAndFinGreaterThanOrEqual(
            usuario.getId(), LocalDate.now()
        );
        if (sancionado) throw new BusinessException("Usuario con
sanción activa.");

        Ejemplar ejemplar = ejemplarRepository.findById(ejemplarId)
            .orElseThrow(() -> new NotFoundException("Ejemplar no
encontrado"));

        if (ejemplar.isReferencia()) throw new
BusinessRuleException("Ejemplar de referencia: no prestable.");
        if (ejemplar.getEstado() != EstadoEjemplar.DISPONIBLE &&
ejemplar.getEstado() != EstadoEjemplar.RESERVADO) {
            throw new BusinessException("Ejemplar no prestable en su
estado actual.");
        }

        LocalDate hoy = LocalDate.now();
        Prestamo prestamo = new Prestamo(usuario, ejemplar, hoy,
hoy.plusDays(DIAS_PRESTAMO));
        ejemplar.setEstado(EstadoEjemplar.PRESTADO);

        ejemplarRepository.save(ejemplar);
        return prestamoRepository.save(prestamo);
    }

    @Transactional
    public void registrarDevolucion(Long ejemplarId) {
        Ejemplar ejemplar = ejemplarRepository.findById(ejemplarId)
            .orElseThrow(() -> new NotFoundException("Ejemplar no
encontrado"));

        // En MVP asumimos 1 préstamo abierto por ejemplar
        Prestamo prestamo = prestamoRepository.findAll().stream()
            .filter(p -> !p.isCerrado() &&

```

```

p.getEjemplar().getId().equals(ejemplarId))
    .findFirst()
    .orElseThrow(() -> new NotFoundException("No hay préstamo
abierto para este ejemplar"));

    prestamo.cerrar();
    ejemplar.setEstado(EstadoEjemplar.DISPONIBLE);

    // sanción por retraso (simple)
    LocalDate hoy = LocalDate.now();
    if (hoy.isAfter(prestamo.getFinPrevisto())) {
        Sancion sancion = new Sancion(
            prestamo.getUsuario(),
            hoy,
            hoy.plusDays(DIAS_SANCION_POR_RETRASO),
            "Devolución fuera de plazo"
        );
        sancionRepository.save(sancion);
    }

    prestamoRepository.save(prestamo);
    ejemplarRepository.save(ejemplar);
}

@Transactional
public Prestamo renovarPrestamo(String externalUserId, Long
prestamoId) {
    Usuario usuario =
usuarioRepository.findByExternalId(externalUserId)
        .orElseThrow(() -> new NotFoundException("Usuario no
encontrado"));

    Prestamo prestamo = prestamoRepository.findById(prestamoId)
        .orElseThrow(() -> new NotFoundException("Préstamo no
encontrado"));

    if (!prestamo.getUsuario().getId().equals(usuario.getId())) {
        throw new BusinessException("No puedes renovar un
préstamo de otro usuario.");
    }
    if (prestamo.isCerrado()) throw new BusinessException("No se
puede renovar un préstamo cerrado.");
    if (prestamo.getRenovaciones() >= MAX_RENOVACIONES) throw new
BusinessRuleException("Límite de renovaciones alcanzado.");

    LocalDate nuevoFin =

```

```

        prestamo.getFinPrevisto().plusDays(DIAS_PRESTAMO);
        prestamo.renovar(nuevoFin);
        return prestamoRepository.save(prestamo);
    }
}

```

MiCuentaService

```

package com.tuorg.biblioteca.service;

import com.tuorg.biblioteca.domain.Usuario;
import com.tuorg.biblioteca.exception.NotFoundException;
import com.tuorg.biblioteca.repository.*;
import org.springframework.stereotype.Service;

@Service
public class MiCuentaService {

    private final UsuarioRepository usuarioRepository;
    private final PrestamoRepository prestamoRepository;
    private final ReservaRepository reservaRepository;
    private final SancionRepository sancionRepository;

    public MiCuentaService(
        UsuarioRepository usuarioRepository,
        PrestamoRepository prestamoRepository,
        ReservaRepository reservaRepository,
        SancionRepository sancionRepository
    ) {
        this.usuarioRepository = usuarioRepository;
        this.prestamoRepository = prestamoRepository;
        this.reservaRepository = reservaRepository;
        this.sancionRepository = sancionRepository;
    }

    public Usuario cargarUsuario(String externalUserId) {
        return usuarioRepository.findByExternalId(externalUserId)
            .orElseThrow(() -> new NotFoundException("Usuario no
encontrado"));
    }

    public Object obtenerDatos(String externalUserId) {
        Usuario u = cargarUsuario(externalUserId);
        // Lo devolvemos como Object aquí; el controller lo mapea a
MiCuentaDto
        return new Object[] {
            prestamoRepository.findByUsuarioId(u.getId()),

```



```

        reservaRepository.findByUsuarioId(u.getId()),
        sancionRepository.findByUsuarioId(u.getId())
    };
}
}

```

7) Controllers (endpoints del MVP)

Helpers para sacar el “sub” del JWT

En controllers, lo más simple:

```

import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.jwt.Jwt;

static String sub(Authentication auth) {
    return ((Jwt) auth.getPrincipal()).getSubject();
}

```

Catálogo

```

package com.tuorg.biblioteca.controller;

import com.tuorg.biblioteca.domain.Ejemplar;
import com.tuorg.biblioteca.dto.catalogo.ObraResumenDto;
import com.tuorg.biblioteca.dto.catalogo.EjemplarDetalleDto;
import com.tuorg.biblioteca.exception.NotFoundException;
import com.tuorg.biblioteca.repository.EjemplarRepository;
import com.tuorg.biblioteca.service.CatalogoService;
import java.util.List;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class CatalogoController {

    private final CatalogoService catalogoService;
    private final EjemplarRepository ejemplarRepository;

    public CatalogoController(CatalogoService catalogoService,
                             EjemplarRepository ejemplarRepository) {
        this.catalogoService = catalogoService;
    }
}

```

```

        this.ejemplarRepository = ejemplarRepository;
    }

    @GetMapping("/catalogo")
    public List<ObraResumenDto> buscar(@RequestParam(required = false)
String query) {
        return catalogoService.buscar(query).stream()
            .map(ObraResumenDto::from)
            .toList();
    }

    @GetMapping("/ejemplares/{id}")
    public EjemplarDetalleDto detalle(@PathVariable Long id) {
        Ejemplar e = ejemplarRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("Ejemplar no
encontrado"));
        return EjemplarDetalleDto.from(e);
    }
}

```

Reservas

```

package com.tuorg.biblioteca.controller;

import com.tuorg.biblioteca.dto.reservas.ReservaCreateRequest;
import com.tuorg.biblioteca.dto.reservas.ReservaDto;
import com.tuorg.biblioteca.service.ReservaService;
import jakarta.validation.Valid;
import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/reservas")
public class ReservaController {

    private final ReservaService reservaService;

    public ReservaController(ReservaService reservaService) {
        this.reservaService = reservaService;
    }

    @PostMapping
    public ReservaDto crear(Authentication auth, @Valid @RequestBody
ReservaCreateRequest req) {
        String sub = ((Jwt) auth.getPrincipal()).getSubject();
        return ReservaDto.from(reservaService.crearReserva(sub,

```

```

req.ejemplarId()));
    }

    @DeleteMapping("/{id}")
    public void cancelar(Authentication auth, @PathVariable Long id) {
        String sub = ((Jwt) auth.getPrincipal()).getSubject();
        reservaService.cancelarReserva(sub, id);
    }
}

```

Mi cuenta

```

package com.tuorg.biblioteca.controller;

import com.tuorg.biblioteca.dto.micuenta.MiCuentaDto;
import com.tuorg.biblioteca.service.MiCuentaService;
import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/mi-cuenta")
public class MiCuentaController {

    private final MiCuentaService miCuentaService;

    public MiCuentaController(MiCuentaService miCuentaService) {
        this.miCuentaService = miCuentaService;
    }

    @GetMapping
    public MiCuentaDto ver(Authentication auth) {
        String sub = ((Jwt) auth.getPrincipal()).getSubject();
        Object[] data = (Object[]) miCuentaService.obtenerDatos(sub);
        return MiCuentaDto.from(data);
    }
}

```

Admin préstamos/devoluciones

```

package com.tuorg.biblioteca.controller.admin;

import com.tuorg.biblioteca.dto.admin.CrearPrestamoRequest;
import com.tuorg.biblioteca.dto.admin.PrestamoAdminDto;
import com.tuorg.biblioteca.dto.admin.RegistrarDevolucionRequest;
import com.tuorg.biblioteca.service.PrestamoService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

```

```

@RestController
@RequestMapping("/api/admin")
public class PrestamoAdminController {

    private final PrestamoService prestamoService;

    public PrestamoAdminController(PrestamoService prestamoService) {
        this.prestamoService = prestamoService;
    }

    @PostMapping("/prestamos")
    public PrestamoAdminDto crearPrestamo(@Valid @RequestBody
    CrearPrestamoRequest req) {
        return
        PrestamoAdminDto.from(prestamoService.registrarPrestamo(req.usuarioId(),
        req.ejemplarId()));
    }

    @PostMapping("/devoluciones")
    public void devolver(@Valid @RequestBody RegistrarDevolucionRequest
    req) {
        prestamoService.registrarDevolucion(req.ejemplarId());
    }
}

```

8) DTOs (ejemplos)

Catalogo

```

package com.tuorg.biblioteca.dto.catalogo;

import com.tuorg.biblioteca.domain.Obra;

public record ObraResumenDto(Long id, String titulo, String autor, String
    materia) {
    public static ObraResumenDto from(Obra o) {
        return new ObraResumenDto(o.getId(), o.getTitulo(), o.getAutor(),
        o.getMateria());
    }
}

```

```

package com.tuorg.biblioteca.dto.catalogo;

import com.tuorg.biblioteca.domain.Ejemplar;

public record EjemplarDetalleDto(
    Long id,
    Long obraId,
    String titulo,
    String autor,
    String estado,
    boolean referencia,
    String ubicacion
) {
    public static EjemplarDetalleDto from(Ejemplar e) {
        return new EjemplarDetalleDto(
            e.getId(),
            e.getObra().getId(),
            e.getObra().getTitulo(),
            e.getObra().getAutor(),
            e.getEstado().name(),
            e.isReferencia(),
            e.getUbicacion()
        );
    }
}

```

Reservas

```

package com.tuorg.biblioteca.dto.reservas;

import jakarta.validation.constraints.NotNull;

public record ReservaCreateRequest(@NotNull Long ejemplarId) {}

package com.tuorg.biblioteca.dto.reservas;

import com.tuorg.biblioteca.domain.Reserva;

public record ReservaDto(Long id, Long ejemplarId, String estado) {
    public static ReservaDto from(Reserva r) {
        return new ReservaDto(r.getId(), r.getEjemplar().getId(),
            r.getEstado().name());
    }
}

```

Admin

```

package com.tuorg.biblioteca.dto.admin;

```

```

import jakarta.validation.constraints.NotNull;

public record CrearPrestamoRequest(@NotNull Long usuarioId, @NotNull Long
ejemplarId) {}
package com.tuorg.biblioteca.dto.admin;

import jakarta.validation.constraints.NotNull;

public record RegistrarDevolucionRequest(@NotNull Long ejemplarId) {}
package com.tuorg.biblioteca.dto.admin;

import com.tuorg.biblioteca.domain.Prestamo;

public record PrestamoAdminDto(Long id, Long usuarioId, Long ejemplarId,
String finPrevisto) {
    public static PrestamoAdminDto from(Prestamo p) {
        return new PrestamoAdminDto(
            p.getId(),
            p.getUsuario().getId(),
            p.getEjemplar().getId(),
            p.getFinPrevisto().toString()
        );
    }
}

```

MiCuentaDto (mínimo)

```

package com.tuorg.biblioteca.dto.micuenta;

import com.tuorg.biblioteca.domain.Prestamo;
import com.tuorg.biblioteca.domain.Reserva;
import com.tuorg.biblioteca.domain.Sancion;
import java.util.List;

public record MiCuentaDto(
    List<PrestamoDto> prestamos,
    List<ReservaDtoSimple> reservas,
    List<SancionDto> sanciones
) {
    @SuppressWarnings("unchecked")
    public static MiCuentaDto from(Object[] data) {
        List<Prestamo> prestamos = (List<Prestamo>) data[0];
        List<Reserva> reservas = (List<Reserva>) data[1];
        List<Sancion> sanciones = (List<Sancion>) data[2];

        return new MiCuentaDto(
            prestamos.stream().map(PrestamoDto::from).toList(),

```

```

        reservas.stream().map(ReservaDtoSimple::from).toList(),
        sanciones.stream().map(SancionDto::from).toList()
    );
}
}
package com.tuorg.biblioteca.dto.micuenta;

import com.tuorg.biblioteca.domain.Prestamo;

public record PrestamoDto(Long id, Long ejemplarId, String finPrevisto,
boolean cerrado, int renovaciones) {
    public static PrestamoDto from(Prestamo p) {
        return new PrestamoDto(p.getId(), p.getEjemplar().getId(),
p.getFinPrevisto().toString(), p.isCerrado(), p.getRenovaciones());
    }
}
package com.tuorg.biblioteca.dto.micuenta;

import com.tuorg.biblioteca.domain.Reserva;

public record ReservaDtoSimple(Long id, Long ejemplarId, String estado) {
    public static ReservaDtoSimple from(Reserva r) {
        return new ReservaDtoSimple(r.getId(), r.getEjemplar().getId(),
r.getEstado().name());
    }
}
package com.tuorg.biblioteca.dto.micuenta;

import com.tuorg.biblioteca.domain.Sancion;

public record SancionDto(Long id, String inicio, String fin, boolean
activa, String causa) {
    public static SancionDto from(Sancion s) {
        return new SancionDto(s.getId(), s.getInicio().toString(),
s.getFin().toString(), s.isActiva(), s.getCausa());
    }
}
}

```

9) Seguridad (JWT + rol bibliotecario)

SecurityConfig

```

package com.tuorg.biblioteca.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/**").permitAll()

            .requestMatchers("/api/admin/**").hasRole("BIBLIOTECARIO")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            );

        return http.build();
    }
}

```

Si tu proveedor mete roles en un claim específico (p.ej. `cognito:groups`), te preparo el `JwtRoleConverter` para mapearlo a `ROLE_BIBLIOTECARIO`. (Es un bloque corto.)

10) Manejo de errores (bonito y consistente)

```

package com.tuorg.biblioteca.exception;

```



```

import java.time.Instant;

public record ApiError(
    Instant timestamp,
    int status,
    String error,
    String message,
    String path
) {}

package com.tuorg.biblioteca.exception;

import jakarta.servlet.http.HttpServletRequest;
import java.time.Instant;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ApiError> notFound(NotFoundException ex,
    HttpServletRequest req) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError(Instant.now(), 404, "Not Found",
ex.getMessage(), req.getRequestURI()));
    }

    @ExceptionHandler(BusinessRuleException.class)
    public ResponseEntity<ApiError> business(BusinessRuleException ex,
    HttpServletRequest req) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(new ApiError(Instant.now(), 409, "Business Rule",
ex.getMessage(), req.getRequestURI()));
    }
}

```

Siguiente paso (te lo dejo mascado)

Dime **qué claim trae el rol** en tu JWT (por ejemplo: roles, groups, cognito:groups, realm_access.roles, etc.). Con eso te

paso el `JwtRoleConverter` para que automáticamente tu `/api/admin/**` funcione con Cognito/SSO.

Y si ya tienes Postgres local, también te paso un `docker-compose.yml` mínimo para levantar la BD en 10 segundos.

You said:

en java 25 se siguen usando getters y setters?

ChatGPT said:

Sí, **se siguen usando getters y setters** en Java 25. No han “desaparecido” ni hay un reemplazo obligatorio del lenguaje.

Lo que ha cambiado es que ahora tienes **más opciones** según el caso:

1) Para *entidades JPA* (lo típico en Spring)

En JPA/Hibernate **siguen siendo muy habituales**:

- **getters** (para lectura, serialización, mapeo, etc.)
- **setters** solo cuando necesitas mutabilidad (o cuando tu estrategia de acceso es por propiedades)

En entidades, mucha gente evita “setters genéricos” y prefiere:

- setters **solo para lo imprescindible**, o
- métodos de dominio tipo `marcarPrestado()`, `cancelar()`, etc.

2) Para *DTOs (requests/responses)*

Aquí, en Java moderno, lo común es usar **records**:

- Son inmutables
- No tienen setters
- No tienen getters “getX()”; tienen **accesores** `x()`

Ejemplo:

```
public record ReservaCreateRequest(Long ejemplarId) {}
```

Se usa así: `req.ejemplarId()`.

3) Para *Value Objects / configuración*

También suele ir bien:

- `record` si es un valor inmutable
 - o clases normales con getters si necesitas frameworks que lo requieran
-

Recomendación práctica para tu proyecto

- **Entidades** (`@Entity`): clase normal, **getters** + mutabilidad controlada (pocos setters o métodos de dominio).
 - **DTOs: records** (casi siempre).
 - **Servicios**: sin getters/setters, solo lógica.
-

Nota importante (Spring/Jackson)

Jackson y Spring hoy trabajan bien con:

- records (DTOs)
 - getters clásicos (entidades/POJOs)
- Así que no estás peleándote con el framework por elegir uno u otro.

Si me dices si estás usando **JPA/Hibernate** sí o sí (lo normal con RDS/Postgres), te digo el patrón exacto recomendado para tus entidades (field access vs property access) para que no te dé guerra.

You said:

quiero el código en inglés

ChatGPT said:

Perfect — from now on I'll write the **code in English** (class names, fields, methods, enums, endpoints, messages), and I'll keep the explanations in Spanish unless you tell me otherwise.

Here's the same answer about **getters/setters in Java 25**, but with **English code examples**:

Do we still use getters and setters in Java 25?

Yes. **Getters and setters are still widely used** in Java 25—especially in Spring/JPA projects.

What's different is that modern Java gives you more choices depending on the layer.

1) JPA Entities (@Entity)

For JPA/Hibernate, the typical approach is still:

- **getters** (common)
- **setters** only when needed

A good modern style is: **avoid “generic setters”** and expose intent through domain methods.

```
@Entity
@Table(name = "copies")
public class Copy {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private CopyStatus status = CopyStatus.AVAILABLE;

    protected Copy() {}
}
```

```
public Long getId() { return id; }
public CopyStatus getStatus() { return status; }

public void markAsLoaned() {
    this.status = CopyStatus.LOANED;
}

public void markAsAvailable() {
    this.status = CopyStatus.AVAILABLE;
}
}
```

This keeps the entity consistent and prevents random state changes.

2) DTOs (API requests/responses)

For DTOs, modern Java typically uses **records**:

- immutable
- no setters
- accessors like `copyId()` instead of `getCopyId()`

```
public record CreateReservationRequest(Long copyId) {}
```

Usage:

```
Long id = request.copyId();
```

3) Services

Services usually don't need getters/setters. They're just classes with behavior.

Recommendation for your project

- **Entities:** regular classes with **getters**, and minimal setters (or better: domain methods).
 - **DTOs: records.**
 - **Controllers/Services:** clean logic, no setters/getters.
-

If you want, I can refactor the skeleton I wrote earlier so **all package names, classes, fields, enums, endpoint paths, and error messages are in English**, consistently (e.g., `CatalogController`, `ReservationService`, `Loan`, `Penalty`, etc.).

You said:

todas las clases y objetos que has escrito antes en ingles

ChatGPT said:

Perfecto. A partir de ahora **todo el código va en inglés**:

- paquetes
- clases
- métodos
- atributos
- enums
- DTOs
- endpoints
- mensajes de error

Por ejemplo:

- `Usuario` → `User`
- `Ejemplar` → `Copy`
- `Prestamo` → `Loan`
- `Reserva` → `Reservation`
- `Sancion` → `Penalty`
- `CatalogoController` → `CatalogController`
- `ReservaService` → `ReservationService`
- `/api/mi-cuenta` → `/api/my-account`

Antes de lanzarme a reescribir todo el esqueleto, confirmo cómo lo quieres:

¿Quieres que te entregue:

1.  **Todo el proyecto reescrito completo en inglés**, listo para copiar/pegar
(dominio, repositorios, servicios, controllers, DTOs, seguridad, excepciones)

o

2.  Primero solo:
 - Dominio (domain)
 - Repositorios (repository)

y luego vamos capa por capa.

Para que veas el estilo que usaré, este sería el cambio de uno de tus enums:

```
public enum CopyStatus {  
    AVAILABLE,  
    RESERVED,  
    LOANED  
}
```

Y una entidad:

```
@Entity  
@Table(name = "copies")  
public class Copy {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @ManyToOne(optional = false, fetch = FetchType.LAZY)  
    private Book book;  
  
    @Enumerated(EnumType.STRING)  
    @Column(nullable = false)  
    private CopyStatus status = CopyStatus.AVAILABLE;  
  
    @Column(nullable = false)
```

```
private boolean referenceOnly;

private String location;

protected Copy() {}

public Long getId() { return id; }
public Book getBook() { return book; }
public CopyStatus getStatus() { return status; }
public boolean isReferenceOnly() { return referenceOnly; }
public String getLocation() { return location; }

public void markAsReserved() {
    this.status = CopyStatus.RESERVED;
}

public void markAsLoaned() {
    this.status = CopyStatus.LOANED;
}

public void markAsAvailable() {
    this.status = CopyStatus.AVAILABLE;
}
}
```

Cuando me confirmes, te preparo el **paquete completo en inglés**, ordenado y coherente con la arquitectura del MVP de tu biblioteca.

You said:

sí

ChatGPT said:

Vale — aquí tienes **todo el esqueleto reescrito completo en inglés** (packages, classes, fields, enums, DTOs, endpoints, error messages). Está pensado como **REST API por capas** (Controller → Service → Repository) y el mismo MVP que te propuse.

Nota rápida: mantengo records para DTOs y **entidades JPA con getters + métodos de dominio** (sin setters genéricos salvo que lo necesites).

Package structure (English)

```
src/main/java/com/yourorg/library/  
  LibraryApplication.java  
  
  security/  
    SecurityConfig.java  
  
  domain/  
    User.java  
    Role.java  
    Book.java  
    Copy.java  
    CopyStatus.java  
    Reservation.java  
    ReservationStatus.java  
    Loan.java  
    Penalty.java  
  
  dto/  
    catalog/  
      BookSummaryDto.java  
      CopyDetailsDto.java  
    reservations/  
      CreateReservationRequest.java  
      ReservationDto.java  
    account/  
      MyAccountDto.java  
      LoanDto.java  
      PenaltyDto.java  
      ReservationSummaryDto.java  
    admin/  
      CreateLoanRequest.java  
      RegisterReturnRequest.java  
      LoanAdminDto.java  
  
  repository/  
    UserRepository.java
```

```
BookRepository.java
CopyRepository.java
ReservationRepository.java
LoanRepository.java
PenaltyRepository.java

service/
  CatalogService.java
  ReservationService.java
  LoanService.java
  MyAccountService.java

controller/
  CatalogController.java
  ReservationController.java
  MyAccountController.java
  admin/
    LoanAdminController.java

exception/
  ApiError.java
  GlobalExceptionHandler.java
  NotFoundException.java
  BusinessException.java
```

DOMAIN

Role.java

```
package com.yourorg.library.domain;

public enum Role {
    USER,
    LIBRARIAN
}
```

CopyStatus.java

```
package com.yourorg.library.domain;

public enum CopyStatus {
    AVAILABLE,
```

```
    RESERVED,  
    LOANED  
}
```

ReservationStatus.java

```
package com.yourorg.library.domain;  
  
public enum ReservationStatus {  
    ACTIVE,  
    CANCELED,  
    FULFILLED,  
    EXPIRED  
}
```

User.java

```
package com.yourorg.library.domain;  
  
import jakarta.persistence.*;  
  
@Entity  
@Table(name = "users")  
public class User {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    // JWT subject (e.g., Cognito/SSO "sub")  
    @Column(nullable = false, unique = true)  
    private String externalId;  
  
    @Enumerated(EnumType.STRING)  
    @Column(nullable = false)  
    private Role role = Role.USER;  
  
    protected User() {}  
  
    public User(String externalId, Role role) {  
        this.externalId = externalId;  
        this.role = role;  
    }  
  
    public Long getId() { return id; }  
    public String getExternalId() { return externalId; }  
    public Role getRole() { return role; }  
}
```

Book.java

```
package com.yourorg.library.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private String author;
    private String subject;

    protected Book() {}

    public Book(String title, String author, String subject) {
        this.title = title;
        this.author = author;
        this.subject = subject;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public String getSubject() { return subject; }
}
```

Copy.java

```
package com.yourorg.library.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "copies")
public class Copy {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

@ManyToOne(optional = false, fetch = FetchType.LAZY)
private Book book;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private CopyStatus status = CopyStatus.AVAILABLE;

@Column(nullable = false)
private boolean referenceOnly = false;

private String location;

protected Copy() {}

public Copy(Book book, boolean referenceOnly, String location) {
    this.book = book;
    this.referenceOnly = referenceOnly;
    this.location = location;
}

public Long getId() { return id; }
public Book getBook() { return book; }
public CopyStatus getStatus() { return status; }
public boolean isReferenceOnly() { return referenceOnly; }
public String getLocation() { return location; }

public void markAsReserved() { this.status = CopyStatus.RESERVED; }
public void markAsLoaned() { this.status = CopyStatus.LOANED; }
public void markAsAvailable() { this.status = CopyStatus.AVAILABLE; }
}

```

Reservation.java

```

package com.yourorg.library.domain;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "reservations")
public class Reservation {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}

```

```

@ManyToOne(optional = false, fetch = FetchType.LAZY)
private User user;

@ManyToOne(optional = false, fetch = FetchType.LAZY)
private Copy copy;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private ReservationStatus status = ReservationStatus.ACTIVE;

@Column(nullable = false)
private Instant createdAt = Instant.now();

protected Reservation() {}

public Reservation(User user, Copy copy) {
    this.user = user;
    this.copy = copy;
}

public Long getId() { return id; }
public User getUser() { return user; }
public Copy getCopy() { return copy; }
public ReservationStatus getStatus() { return status; }
public Instant getCreatedAt() { return createdAt; }

public void cancel() { this.status = ReservationStatus.CANCELED; }
public void fulfill() { this.status = ReservationStatus.FULFILLED; }
}

```

Loan.java

```

package com.yourorg.library.domain;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "loans")
public class Loan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private User user;

```

```

@ManyToOne(optional = false, fetch = FetchType.LAZY)
private Copy copy;

@Column(nullable = false)
private LocalDate startDate;

@Column(nullable = false)
private LocalDate dueDate;

@Column(nullable = false)
private int renewals = 0;

@Column(nullable = false)
private boolean closed = false;

protected Loan() {}

public Loan(User user, Copy copy, LocalDate startDate, LocalDate
dueDate) {
    this.user = user;
    this.copy = copy;
    this.startDate = startDate;
    this.dueDate = dueDate;
}

public Long getId() { return id; }
public User getUser() { return user; }
public Copy getCopy() { return copy; }
public LocalDate getStartDate() { return startDate; }
public LocalDate getDueDate() { return dueDate; }
public int getRenewals() { return renewals; }
public boolean isClosed() { return closed; }

public void renew(LocalDate newDueDate) {
    this.dueDate = newDueDate;
    this.renewals++;
}

public void close() { this.closed = true; }
}

```

Penalty.java

```

package com.yourorg.library.domain;

import jakarta.persistence.*;

```

```

import java.time.LocalDate;

@Entity
@Table(name = "penalties")
public class Penalty {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    private User user;

    @Column(nullable = false)
    private LocalDate startDate;

    @Column(nullable = false)
    private LocalDate endDate;

    @Column(nullable = false)
    private boolean active = true;

    private String reason;

    protected Penalty() {}

    public Penalty(User user, LocalDate startDate, LocalDate endDate,
String reason) {
        this.user = user;
        this.startDate = startDate;
        this.endDate = endDate;
        this.reason = reason;
        this.active = true;
    }

    public Long getId() { return id; }
    public User getUser() { return user; }
    public LocalDate getStartDate() { return startDate; }
    public LocalDate getEndDate() { return endDate; }
    public boolean isActive() { return active; }
    public String getReason() { return reason; }

    public void deactivate() { this.active = false; }
}

```

REPOSITORIES

UserRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.User;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByExternalId(String externalId);
}
```

BookRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface BookRepository extends JpaRepository<Book, Long>,
JpaSpecificationExecutor<Book> {}
```

CopyRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.Copy;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CopyRepository extends JpaRepository<Copy, Long> {}
```

ReservationRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.Reservation;
import com.yourorg.library.domain.ReservationStatus;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ReservationRepository extends JpaRepository<Reservation,
Long> {
```

```
        List<Reservation> findByUserId(Long userId);
        List<Reservation> findByUserIdAndStatus(Long userId,
ReservationStatus status);
    }
```

LoanRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.Loan;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface LoanRepository extends JpaRepository<Loan, Long> {
    List<Loan> findByUserId(Long userId);
}
```

PenaltyRepository.java

```
package com.yourorg.library.repository;

import com.yourorg.library.domain.Penalty;
import java.time.LocalDate;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PenaltyRepository extends JpaRepository<Penalty, Long> {
    List<Penalty> findByUserId(Long userId);

    boolean existsByUserIdAndActiveIsTrueAndEndDateGreaterThanOrEqual(Long
userId, LocalDate today);
}
```

DTOs

dto/catalog/BookSummaryDto.java

```
package com.yourorg.library.dto.catalog;

import com.yourorg.library.domain.Book;

public record BookSummaryDto(Long id, String title, String author, String
subject) {
```

```

        public static BookSummaryDto from(Book b) {
            return new BookSummaryDto(b.getId(), b.getTitle(), b.getAuthor(),
b.getSubject());
        }
    }
}

```

dto/catalog/CopyDetailsDto.java

```

package com.yourorg.library.dto.catalog;

import com.yourorg.library.domain.Copy;

public record CopyDetailsDto(
    Long id,
    Long bookId,
    String title,
    String author,
    String status,
    boolean referenceOnly,
    String location
) {
    public static CopyDetailsDto from(Copy c) {
        return new CopyDetailsDto(
            c.getId(),
            c.getBook().getId(),
            c.getBook().getTitle(),
            c.getBook().getAuthor(),
            c.getStatus().name(),
            c.isReferenceOnly(),
            c.getLocation()
        );
    }
}

```

dto/reservations/CreateReservationRequest.java

```

package com.yourorg.library.dto.reservations;

import jakarta.validation.constraints.NotNull;

public record CreateReservationRequest(@NotNull Long copyId) {}

```

dto/reservations/ReservationDto.java

```

package com.yourorg.library.dto.reservations;

import com.yourorg.library.domain.Reservation;

```

```

public record ReservationDto(Long id, Long copyId, String status) {
    public static ReservationDto from(Reservation r) {
        return new ReservationDto(r.getId(), r.getCopy().getId(),
r.getStatus().name());
    }
}

```

dto/admin/CreateLoanRequest.java

```

package com.yourorg.library.dto.admin;

import jakarta.validation.constraints.NotNull;

public record CreateLoanRequest(@NotNull Long userId, @NotNull Long
copyId) {}

```

dto/admin/RegisterReturnRequest.java

```

package com.yourorg.library.dto.admin;

import jakarta.validation.constraints.NotNull;

public record RegisterReturnRequest(@NotNull Long copyId) {}

```

dto/admin/LoanAdminDto.java

```

package com.yourorg.library.dto.admin;

import com.yourorg.library.domain.Loan;

public record LoanAdminDto(Long id, Long userId, Long copyId, String
dueDate) {
    public static LoanAdminDto from(Loan l) {
        return new LoanAdminDto(
            l.getId(),
            l.getUser().getId(),
            l.getCopy().getId(),
            l.getDueDate().toString()
        );
    }
}

```

dto/account/LoanDto.java

```

package com.yourorg.library.dto.account;

import com.yourorg.library.domain.Loan;

public record LoanDto(Long id, Long copyId, String dueDate, boolean

```

```

closed, int renewals) {
    public static LoanDto from(Loan l) {
        return new LoanDto(
            l.getId(),
            l.getCopy().getId(),
            l.getDueDate().toString(),
            l.isClosed(),
            l.getRenewals()
        );
    }
}

```

dto/account/ReservationSummaryDto.java

```

package com.yourorg.library.dto.account;

import com.yourorg.library.domain.Reservation;

public record ReservationSummaryDto(Long id, Long copyId, String status)
{
    public static ReservationSummaryDto from(Reservation r) {
        return new ReservationSummaryDto(r.getId(), r.getCopy().getId(),
r.getStatus().name());
    }
}

```

dto/account/PenaltyDto.java

```

package com.yourorg.library.dto.account;

import com.yourorg.library.domain.Penalty;

public record PenaltyDto(Long id, String startDate, String endDate,
boolean active, String reason) {
    public static PenaltyDto from(Penalty p) {
        return new PenaltyDto(
            p.getId(),
            p.getStartDate().toString(),
            p.getEndDate().toString(),
            p.isActive(),
            p.getReason()
        );
    }
}

```

dto/account/MyAccountDto.java

```

package com.yourorg.library.dto.account;

import com.yourorg.library.domain.Loan;
import com.yourorg.library.domain.Penalty;
import com.yourorg.library.domain.Reservation;
import java.util.List;

public record MyAccountDto(
    List<LoanDto> loans,
    List<ReservationSummaryDto> reservations,
    List<PenaltyDto> penalties
) {
    @SuppressWarnings("unchecked")
    public static MyAccountDto from(Object[] data) {
        List<Loan> loans = (List<Loan>) data[0];
        List<Reservation> reservations = (List<Reservation>) data[1];
        List<Penalty> penalties = (List<Penalty>) data[2];

        return new MyAccountDto(
            loans.stream().map(LoanDto::from).toList(),
            reservations.stream().map(ReservationSummaryDto::from).toList(),
            penalties.stream().map(PenaltyDto::from).toList()
        );
    }
}

```

EXCEPTIONS

NotFoundException.java

```

package com.yourorg.library.exception;

public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) { super(message); }
}

```

BusinessRuleException.java

```

package com.yourorg.library.exception;

public class BusinessRuleException extends RuntimeException {

```

```
    public BusinessException(String message) { super(message); }  
}
```

ApiError.java

```
package com.yourorg.library.exception;  
  
import java.time.Instant;  
  
public record ApiError(  
    Instant timestamp,  
    int status,  
    String error,  
    String message,  
    String path  
) {}
```

GlobalExceptionHandler.java

```
package com.yourorg.library.exception;  
  
import jakarta.servlet.http.HttpServletRequest;  
import java.time.Instant;  
import org.springframework.http.*;  
import org.springframework.web.bind.annotation.*;  
  
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(NotFoundException.class)  
    public ResponseEntity<ApiError> handleNotFound(NotFoundException ex,  
        HttpServletRequest req) {  
        return ResponseEntity.status(HttpStatus.NOT_FOUND)  
            .body(new ApiError(Instant.now(), 404, "Not Found",  
ex.getMessage(), req.getRequestURI()));  
    }  
  
    @ExceptionHandler(BusinessRuleException.class)  
    public ResponseEntity<ApiError>  
handleBusinessRule(BusinessRuleException ex, HttpServletRequest req) {  
        return ResponseEntity.status(HttpStatus.CONFLICT)  
            .body(new ApiError(Instant.now(), 409, "Business Rule  
Violation", ex.getMessage(), req.getRequestURI()));  
    }  
}
```

SERVICES

CatalogService.java

```
package com.yourorg.library.service;

import com.yourorg.library.domain.Book;
import com.yourorg.library.repository.BookRepository;
import java.util.List;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;

@Service
public class CatalogService {

    private final BookRepository bookRepository;

    public CatalogService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public List<Book> search(String query) {
        if (query == null || query.isBlank()) {
            return bookRepository.findAll();
        }

        String q = "%" + query.trim().toLowerCase() + "%";

        Specification<Book> spec = (root, cq, cb) ->
            cb.or(
                cb.like(cb.lower(root.get("title")), q),
                cb.like(cb.lower(root.get("author")), q),
                cb.like(cb.lower(root.get("subject")), q)
            );

        return bookRepository.findAll(spec);
    }
}
```

ReservationService.java

```
package com.yourorg.library.service;
```



```

import com.yourorg.library.domain.*;
import com.yourorg.library.exception.BusinessRuleException;
import com.yourorg.library.exception.NotFoundException;
import com.yourorg.library.repository.*;
import java.time.LocalDate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ReservationService {

    private final UserRepository userRepository;
    private final CopyRepository copyRepository;
    private final ReservationRepository reservationRepository;
    private final PenaltyRepository penaltyRepository;

    public ReservationService(
        UserRepository userRepository,
        CopyRepository copyRepository,
        ReservationRepository reservationRepository,
        PenaltyRepository penaltyRepository
    ) {
        this.userRepository = userRepository;
        this.copyRepository = copyRepository;
        this.reservationRepository = reservationRepository;
        this.penaltyRepository = penaltyRepository;
    }

    @Transactional
    public Reservation createReservation(String externalUserId, Long
copyId) {
        User user = userRepository.findByExternalId(externalUserId)
            .orElseThrow(() -> new NotFoundException("User not found."));

        boolean hasActivePenalty =
penaltyRepository.existsByUserIdAndActiveIsTrueAndEndDateGreaterThanOrEqual
(
            user.getId(), LocalDate.now()
        );
        if (hasActivePenalty) {
            throw new BusinessRuleException("You cannot reserve items
while you have an active penalty.");
        }

        Copy copy = copyRepository.findById(copyId)
            .orElseThrow(() -> new NotFoundException("Copy not found."));
    }

```

```

        if (copy.isReferenceOnly()) {
            throw new BusinessException("Reference-only copies cannot
be reserved.");
        }
        if (copy.getStatus() != CopyStatus.AVAILABLE) {
            throw new BusinessException("Copy is not available.");
        }

        Reservation reservation = new Reservation(user, copy);
        copy.markAsReserved();

        copyRepository.save(copy);
        return reservationRepository.save(reservation);
    }

    @Transactional
    public void cancelReservation(String externalUserId, Long
reservationId) {
        User user = userRepository.findByExternalId(externalUserId)
            .orElseThrow(() -> new NotFoundException("User not found."));

        Reservation reservation =
reservationRepository.findById(reservationId)
            .orElseThrow(() -> new NotFoundException("Reservation not
found."));

        if (!reservation.getUser().getId().equals(user.getId())) {
            throw new BusinessException("You cannot cancel another
user's reservation.");
        }
        if (reservation.getStatus() != ReservationStatus.ACTIVE) {
            throw new BusinessException("Only active reservations can
be canceled.");
        }

        reservation.cancel();
        reservation.getCopy().markAsAvailable();

        reservationRepository.save(reservation);
        copyRepository.save(reservation.getCopy());
    }
}

```

LoanService.java

```

package com.yourorg.library.service;

import com.yourorg.library.domain.*;
import com.yourorg.library.exception.BusinessRuleException;
import com.yourorg.library.exception.NotFoundException;
import com.yourorg.library.repository.*;
import java.time.LocalDate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class LoanService {

    private final UserRepository userRepository;
    private final CopyRepository copyRepository;
    private final LoanRepository loanRepository;
    private final PenaltyRepository penaltyRepository;

    // MVP constants (Later move to DB config)
    private static final int LOAN_DAYS = 14;
    private static final int LATE_RETURN_PENALTY_DAYS = 7;
    private static final int MAX_RENEWALS = 1;

    public LoanService(
        UserRepository userRepository,
        CopyRepository copyRepository,
        LoanRepository loanRepository,
        PenaltyRepository penaltyRepository
    ) {
        this.userRepository = userRepository;
        this.copyRepository = copyRepository;
        this.loanRepository = loanRepository;
        this.penaltyRepository = penaltyRepository;
    }

    @Transactional
    public Loan createLoan(Long userId, Long copyId) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new NotFoundException("User not found."));

        boolean hasActivePenalty =
penaltyRepository.existsByUserIdAndActiveIsTrueAndEndDateGreaterThanOrEqualTo
(
        user.getId(), LocalDate.now()
    );
        if (hasActivePenalty) {

```

```

        throw new BusinessException("User has an active
penalty.");
    }

    Copy copy = copyRepository.findById(copyId)
        .orElseThrow(() -> new NotFoundException("Copy not found."));

    if (copy.isReferenceOnly()) {
        throw new BusinessException("Reference-only copies cannot
be loaned.");
    }
    if (copy.getStatus() == CopyStatus.LOANED) {
        throw new BusinessException("Copy is already loaned.");
    }
    // If RESERVED, MVP accepts it (real rule may require "reserved
by same user")
    if (copy.getStatus() != CopyStatus.AVAILABLE && copy.getStatus()
!= CopyStatus.RESERVED) {
        throw new BusinessException("Copy cannot be loaned in its
current status.");
    }

    LocalDate today = LocalDate.now();
    Loan loan = new Loan(user, copy, today,
today.plusDays(LOAN_DAYS));
    copy.markAsLoaned();

    copyRepository.save(copy);
    return loanRepository.save(loan);
}

@Transactional
public void registerReturn(Long copyId) {
    Copy copy = copyRepository.findById(copyId)
        .orElseThrow(() -> new NotFoundException("Copy not found."));

    Loan openLoan = loanRepository.findAll().stream()
        .filter(l -> !l.isClosed() &&
l.getCopy().getId().equals(copyId))
        .findFirst()
        .orElseThrow(() -> new NotFoundException("No open loan found
for this copy."));

    openLoan.close();
    copy.markAsAvailable();
}

```

```

        LocalDate today = LocalDate.now();
        if (today.isAfter(openLoan.getDueDate())) {
            Penalty penalty = new Penalty(
                openLoan.getUser(),
                today,
                today.plusDays(LATE_RETURN_PENALTY_DAYS),
                "Late return"
            );
            penaltyRepository.save(penalty);
        }

        loanRepository.save(openLoan);
        copyRepository.save(copy);
    }

    @Transactional
    public Loan renewLoan(String externalUserId, Long loanId) {
        User user = userRepository.findByExternalId(externalUserId)
            .orElseThrow(() -> new NotFoundException("User not found."));

        Loan loan = loanRepository.findById(loanId)
            .orElseThrow(() -> new NotFoundException("Loan not found."));

        if (!loan.getUser().getId().equals(user.getId())) {
            throw new BusinessException("You cannot renew another
user's loan.");
        }
        if (loan.isClosed()) {
            throw new BusinessException("Closed loans cannot be
renewed.");
        }
        if (loan.getRenewals() >= MAX_RENEWALS) {
            throw new BusinessException("Maximum renewals reached.");
        }

        loan.renew(loan.getDueDate().plusDays(LOAN_DAYS));
        return loanRepository.save(loan);
    }
}

```

MyAccountService.java

```

package com.yourorg.library.service;

import com.yourorg.library.domain.User;
import com.yourorg.library.exception.NotFoundException;
import com.yourorg.library.repository.*;

```

```

import org.springframework.stereotype.Service;

@Service
public class MyAccountService {

    private final UserRepository userRepository;
    private final LoanRepository loanRepository;
    private final ReservationRepository reservationRepository;
    private final PenaltyRepository penaltyRepository;

    public MyAccountService(
        UserRepository userRepository,
        LoanRepository loanRepository,
        ReservationRepository reservationRepository,
        PenaltyRepository penaltyRepository
    ) {
        this.userRepository = userRepository;
        this.loanRepository = loanRepository;
        this.reservationRepository = reservationRepository;
        this.penaltyRepository = penaltyRepository;
    }

    public Object[] getAccountData(String externalUserId) {
        User user = userRepository.findByExternalId(externalUserId)
            .orElseThrow(() -> new NotFoundException("User not found."));

        return new Object[] {
            loanRepository.findById(user.getId()),
            reservationRepository.findById(user.getId()),
            penaltyRepository.findById(user.getId())
        };
    }
}

```

CONTROLLERS (English endpoints)

CatalogController.java

```

package com.yourorg.library.controller;

import com.yourorg.library.domain.Copy;

```

```

import com.yourorg.library.dto.catalog.BookSummaryDto;
import com.yourorg.library.dto.catalog.CopyDetailsDto;
import com.yourorg.library.exception.NotFoundException;
import com.yourorg.library.repository.CopyRepository;
import com.yourorg.library.service.CatalogService;
import java.util.List;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class CatalogController {

    private final CatalogService catalogService;
    private final CopyRepository copyRepository;

    public CatalogController(CatalogService catalogService,
CopyRepository copyRepository) {
        this.catalogService = catalogService;
        this.copyRepository = copyRepository;
    }

    @GetMapping("/catalog")
    public List<BookSummaryDto> search(@RequestParam(required = false)
String query) {
        return catalogService.search(query).stream()
            .map(BookSummaryDto::from)
            .toList();
    }

    @GetMapping("/copies/{id}")
    public CopyDetailsDto getCopyDetails(@PathVariable Long id) {
        Copy copy = copyRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("Copy not found."));
        return CopyDetailsDto.from(copy);
    }
}

```

ReservationController.java

```

package com.yourorg.library.controller;

import com.yourorg.library.dto.reservations.CreateReservationRequest;
import com.yourorg.library.dto.reservations.ReservationDto;
import com.yourorg.library.service.ReservationService;
import jakarta.validation.Valid;
import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.jwt.Jwt;

```

```

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/reservations")
public class ReservationController {

    private final ReservationService reservationService;

    public ReservationController(ReservationService reservationService) {
        this.reservationService = reservationService;
    }

    @PostMapping
    public ReservationDto create(Authentication auth, @Valid @RequestBody
CreateReservationRequest req) {
        String subject = ((Jwt) auth.getPrincipal()).getSubject();
        return
ReservationDto.from(reservationService.createReservation(subject,
req.copyId()));
    }

    @DeleteMapping("/{id}")
    public void cancel(Authentication auth, @PathVariable Long id) {
        String subject = ((Jwt) auth.getPrincipal()).getSubject();
        reservationService.cancelReservation(subject, id);
    }
}

```

MyAccountController.java

```

package com.yourorg.library.controller;

import com.yourorg.library.dto.account.MyAccountDto;
import com.yourorg.library.service.LoanService;
import com.yourorg.library.service.MyAccountService;
import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/my-account")
public class MyAccountController {

    private final MyAccountService myAccountService;
    private final LoanService loanService;

    public MyAccountController(MyAccountService myAccountService,

```



```

LoanService loanService) {
    this.myAccountService = myAccountService;
    this.loanService = loanService;
}

@GetMapping
public MyAccountDto get(Authentication auth) {
    String subject = ((Jwt) auth.getPrincipal()).getSubject();
    return
MyAccountDto.from(myAccountService.getAccountData(subject));
}

@PostMapping("/loans/{loanId}/renew")
public void renew(Authentication auth, @PathVariable Long loanId) {
    String subject = ((Jwt) auth.getPrincipal()).getSubject();
    loanService.renewLoan(subject, loanId);
}
}

```

admin/LoanAdminController.java

```

package com.yourorg.library.controller.admin;

import com.yourorg.library.dto.admin.CreateLoanRequest;
import com.yourorg.library.dto.admin.LoanAdminDto;
import com.yourorg.library.dto.admin.RegisterReturnRequest;
import com.yourorg.library.service.LoanService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/admin")
public class LoanAdminController {

    private final LoanService loanService;

    public LoanAdminController(LoanService loanService) {
        this.loanService = loanService;
    }

    @PostMapping("/loans")
    public LoanAdminDto createLoan(@Valid @RequestBody CreateLoanRequest
req) {
        return LoanAdminDto.from(loanService.createLoan(req.userId(),
req.copyId()));
    }
}

```

```

    @PostMapping("/returns")
    public void registerReturn(@Valid @RequestBody RegisterReturnRequest
req) {
        loanService.registerReturn(req.copyId());
    }
}

```

SECURITY

SecurityConfig.java

```

package com.yourorg.library.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.Enable
eMethodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("LIBRARIAN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 ->
oauth2.jwt(Customizer.withDefaults()));

        return http.build();
    }
}

```

```
}  
}
```

Important note (roles claim)

Right now this assumes Spring will see `ROLE_LIBRARIAN`. In real Cognito/SSO, roles often come in a claim like:

- `cognito:groups`
- `groups`
- `roles`
- `realm_access.roles`

If you tell me **which claim contains the role/group**, I'll give you a small `JwtAuthenticationConverter` so `hasRole("LIBRARIAN")` works correctly.

If you want, next I can also give you:

- a **Postgres docker-compose.yml** (local dev)
- **seed data** (a few books/copies)
- and the **proper query** in `LoanRepository` to find the open loan by copy (instead of the `findAll().stream()` MVP shortcut).

You said:

.DS_Store

File

config.py

Python

main.py

Python

requirements.txt

Document

test.py

Python

users.py

Python

tengo un archivo que es un fake_oidc, que quiero usar como servidor en mi aplicación spring boot. Como puedo desplegar esto en AWS

ChatGPT said:

Thought for 21s

Sí puedes desplegar tu fake_oidc en AWS sin complicarte mucho. Por lo que has subido, es un **provider OIDC falso hecho con FastAPI + Uvicorn**

main

y ahora mismo está configurado para correr en localhost:8000

config

, así que en AWS tendrás que cambiar el issuer y mover secretos a variables de entorno.

Abajo te dejo **2 formas** (la 1 es la más simple para MVP):

Opción 1 (recomendada MVP): Elastic Beanstalk (Python) + ALB (HTTP/HTTPS)

1) Cambia config.py para usar variables de entorno

Tu OIDC_ISSUER está fijo a http://localhost:8000

config

. En AWS debe ser tu URL pública.

config.py (English code):

```
import os

OIDC_ISSUER = os.getenv("OIDC_ISSUER", "http://localhost:8000")
CLIENT_ID = os.getenv("CLIENT_ID", "tfg-client")
CLIENT_SECRET = os.getenv("CLIENT_SECRET", "tfg-secret")
JWT_SECRET = os.getenv("JWT_SECRET", "super-secret-key")
JWT_ALG = os.getenv("JWT_ALG", "HS256")
```

Así podrás configurar todo desde Elastic Beanstalk sin tocar código.

2) Asegura que Uvicorn escuche en 0.0.0.0 y un puerto dinámico

Beanstalk normalmente expone `PORT` (o usas 8000 fijo). Para evitar sorpresas, usa el que te dé EB:

Crea un archivo `Procfile` en la raíz:

Procfile (English code):

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

Tu app ya se llama `app = FastAPI(...)` en `main.py`

`main`

, así que `main:app` está bien.

3) Despliegue con EB CLI (pasos)

1. Instala EB CLI (local):
 - macOS: `brew install awsebcli`
2. En la carpeta del `fake_oidc`:

```
eb init
```

 - Platform: **Python**
 - Region: la que uses
3. Crea entorno:

```
eb create fake-oidc-env
```
4. Configura variables de entorno (muy importante):

```
eb setenv \  
OIDC_ISSUER="https://<your-domain-or-eb-url>" \  
CLIENT_ID="tfg-client" \  
CLIENT_SECRET="tfg-secret" \  
JWT_SECRET="<a-strong-secret>" \  
JWT_ALG="HS256"
```

5. Deploy:

```
eb deploy
```

4) HTTPS (muy recomendado para OIDC)

OIDC en serio suele ir por HTTPS. En EB:

- pon el entorno detrás de **Application Load Balancer**
- crea un certificado en **ACM**
- configura listener 443

Luego tu `OIDC_ISSUER` debería ser:
`https://fake-oidc.<tu-dominio>`

Opción 2: Docker + ECS Fargate (más “cloud native”)

Si prefieres contenedor (más portable):

Dockerfile

Dockerfile (English code):

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
ENV PYTHONUNBUFFERED=1

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Tu `requirements.txt` ya tiene `fastapi`, `uvicorn`, `python-jose`, etc.

`requirements`

Luego:

- build
- push a **ECR**

- crea **ECS Fargate Service** detrás de un **ALB**
 - configura variables de entorno en el Task Definition (OIDC_ISSUER, JWT_SECRET...)
-

Cosas clave para que funcione con tu Spring Boot

1) Cambiar el issuer y los redirect_uri

En tu servidor, el `iss` del `id_token` sale de `OIDC_ISSUER`

main

. Si Spring valida issuer, tiene que coincidir 100%.

Además, en `/authorize` tú rediriges a `redirect_uri` con el `code`

main

, así que en AWS también tienes que actualizar tus redirect URIs (frontend y/o callback).

2) Ojo: tu “fake OIDC” no expone discovery ni JWKS

Tu implementación ahora mismo tiene `/authorize`, `/token`, `/userinfo`

main

pero **no** tiene:

- `/.well-known/openid-configuration`
- `/jwks.json`

Si en Spring estás usando `issuer-uri`, Spring normalmente intenta discovery. Sin esos endpoints, puede fallar.

Soluciones típicas:

- **A)** Implementar discovery + jwks (lo más OIDC-correcto)

- **B)** Configurar Spring para validar JWT de otra forma (por ejemplo con una **secret key** si mantienes HS256)

Si me dices si tu Spring usa:

- `spring.security.oauth2.resourceserver.jwt.issuer-uri`
o
 - `spring.security.oauth2.client.provider...`
- te digo cuál de las dos rutas te conviene y te paso el código exacto (en inglés).
-

Recomendación final (MVP)

Para tu caso (TFG/MVP):

- **Elastic Beanstalk** para el fake_oidc (rápido)
- variables de entorno para secretos (mínimo aceptable)
- HTTPS si vas a integrar "OIDC realista"

Si quieres, te preparo:

1. el `/.well-known/openid-configuration` y un `jwks.json` mínimo (en inglés), o
2. la configuración de Spring Boot para validar HS256 con `JWT_SECRET`.

Dime cuál estás usando en Spring: **OAuth2 Login** (client) o **Resource Server** (validar JWT en API).

Sources

You said:

quiero desplegarlo usando el free tier de aws lo maximo posible

ChatGPT said:

Thought for 21s

Si quieres usar **el Free Tier al máximo**, la clave es evitar servicios que cobran "por hora" aunque tengas poco tráfico (sobre todo **Load Balancers**). Con un *fake OIDC* (FastAPI/Uvicorn) lo más barato/casi gratis suele ser:

La opción más “Free Tier friendly”: EC2 Free Tier (t2.micro/t3.micro) + Nginx + Let's Encrypt

Por qué

- El Free Tier clásico incluye **750 horas/mes** de una instancia **t2.micro** (y también suele aplicar a t3.micro según región/oferta) durante 12 meses para cuentas nuevas.
 - **Elastic Load Balancing (ALB)** no es gratis: tiene coste por hora + LCU.
 - Elastic Beanstalk *como servicio* no cobra extra, pero lo que crea debajo (EC2, EBS, LB, etc.) sí puede costar.
-

Paso a paso (EC2)

1) Lanza una instancia EC2 Free Tier

1. En AWS → **EC2** → **Launch instance**
2. AML: **Amazon Linux 2023** o **Ubuntu**
3. Tipo: **t2.micro** (o t3.micro si aparece como Free Tier en tu cuenta)
4. Storage: mantente dentro de Free Tier (típicamente **hasta 30 GB EBS**; depende de tu plan)
5. Security Group:
 - Inbound: 22 (tu IP), 80 (0.0.0.0/0), 443 (0.0.0.0/0)

Consejo: abre el 22 solo a tu IP.

2) Sube tu proyecto y ejecuta el servidor

En la instancia:

Instala dependencias

Ubuntu

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nginx
```

Amazon Linux

```
sudo dnf update -y
sudo dnf install -y python3 nginx
```

Despliega tu app

Copia tu carpeta `fake_oidc` (scp/git). Luego:

```
cd fake_oidc
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Variables de entorno (muy importante)

Tu `OIDC_ISSUER` ahora está en localhost, y en AWS debe ser tu URL pública. En tu código ya conviene leer env vars (como te sugerí antes). Por ejemplo:

```
export OIDC_ISSUER="https://YOUR_DOMAIN"
export CLIENT_ID="tfg-client"
export CLIENT_SECRET="tfg-secret"
export JWT_SECRET="CHANGE_ME_STRONG"
export JWT_ALG="HS256"
```

Ejecuta Uvicorn en localhost (detrás de Nginx)

```
uvicorn main:app --host 127.0.0.1 --port 8000
```

3) Pon Nginx como reverse proxy (puertos 80/443)

Crea `/etc/nginx/sites-available/fake-oidc` (Ubuntu) o config equivalente en Amazon Linux.

Ejemplo (English config):

```
server {
    listen 80;
    server_name YOUR_DOMAIN;

    location / {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

```
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}
```

Actívalo y reinicia:

```
sudo ln -s /etc/nginx/sites-available/fake-oidc /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

4) HTTPS gratis (Let's Encrypt)

Necesitas un dominio... pero puedes hacerlo sin pagar Route53

Para minimizar coste:

- Puedes usar un dominio que ya tengas, o
- usar un "dominio gratis basado en IP" tipo `nip.io/sslip.io` (ej: `18-xxx-xxx-xxx.nip.io` apunta a tu IP). Esto suele funcionar muy bien para proyectos.

Luego:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d YOUR_DOMAIN
```

Esto te deja HTTPS sin pagar certificados (Let's Encrypt es gratis).

(ACM también ofrece certificados públicos sin coste, pero solo para servicios integrados con ACM; igualmente el LB te costaría).

5) Arranque automático (systemd)

Crea `/etc/systemd/system/fake-oidc.service`:

```
[Unit]
Description=Fake OIDC Provider (FastAPI)
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/fake_oidc
Environment=OIDC_ISSUER=https://YOUR_DOMAIN
Environment=CLIENT_ID=tf-g-client
Environment=CLIENT_SECRET=tf-g-secret
Environment=JWT_SECRET=CHANGE_ME_STRONG
Environment=JWT_ALG=HS256
ExecStart=/home/ec2-user/fake_oidc/.venv/bin/uvicorn main:app --host
127.0.0.1 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

Activa:

```
sudo systemctl daemon-reload
sudo systemctl enable fake-oidc
sudo systemctl start fake-oidc
```

¿Y Elastic Beanstalk en Free Tier?

Se puede, pero para "máximo free" **ten cuidado**:

- EB no cobra extra, pero **si te crea un Load Balancer** te saldrá de Free Tier.
- Si lo haces con EB: usa **Single instance**, sin LB.

Extra importante para Spring Boot (para que no te falle OIDC)

Tu fake OIDC tiene `/authorize`, `/token`, `/userinfo`, pero típicamente Spring con `issuer-uri` intenta discovery (`/.well-known/openid-configuration`). Si lo estás usando como **OAuth2 Client** (login), casi seguro necesitarás:

- `/.well-known/openid-configuration`
- y probablemente `jwks_uri` (o configurar Spring para HS256/secret si estás como Resource Server)

Si me dices cuál de estos estás usando en Spring:

1. `spring.security.oauth2.client...` (login) 
2. `spring.security.oauth2.resourceserver.jwt...` (API valida JWT)

te digo exactamente qué endpoints te faltan y te paso el código (en inglés) para que funcione perfecto.

Recomendación final (para “lo más Free Tier posible”)

✓ **EC2 t2.micro + Nginx + Let's Encrypt + nip.io (sin Route53)**

⊖ Evitar **ALB / Fargate / App Runner** si el objetivo es pagar ~0.

¿Tu cuenta AWS es “nueva” (con Free Tier completo) o ya antigua? Esto cambia mucho lo que realmente sale gratis.

You said:

solo quiero desplegar el servidor fake oidc, de momento no quiero desplegar mi programa

ChatGPT said:

Thought for 13s

Perfecto: si **solo** quieres desplegar *el servidor fake OIDC* y gastar **lo mínimo posible (Free Tier al máximo)**, lo mejor es **EC2 Free Tier (t2.micro)** y **sin Load Balancer** (porque un ALB tiene coste por hora).

Abajo tienes un “camino feliz” simple, barato y fácil de mantener.

Opción recomendada (más “free-tier friendly”): EC2 t2.micro + systemd + (opcional) Nginx + HTTPS

0) Qué vas a conseguir

- Tu fake OIDC corriendo 24/7 en una instancia Free Tier (una sola).
- Acceso por URL pública.
- Si quieres, HTTPS gratis con Let’s Encrypt.

EC2 Free Tier incluye **750 horas/mes** de una instancia **t2.micro** (durante el periodo Free Tier de la cuenta).

1) Crea la instancia EC2 (t2.micro)

En AWS → **EC2** → **Launch instance**

- AMI: Ubuntu 22.04 (o Amazon Linux)
 - Instance type: **t2.micro**
 - Security Group (Inbound):
 - **22** (solo tu IP)
 - **80** (0.0.0.0/0) *(si usarás Nginx/HTTPS)*
 - **443** (0.0.0.0/0) *(si usarás HTTPS)*
 - Si NO quieres Nginx: abre **8000** (0.0.0.0/0) y listo (menos recomendable).
-

2) Sube el fake_oidc a la instancia

En tu local, copia los archivos a EC2 (ejemplo con scp):

```
scp -i your-key.pem -r ./fake_oidc  
ubuntu@<EC2_PUBLIC_IP>:/home/ubuntu/fake_oidc
```

3) Instala Python y dependencias

En la instancia (Ubuntu):

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
```

Crea venv e instala:

```
cd ~/fake_oidc
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

4) Configura variables de entorno (issuer, secretos)

Tu código tiene `OIDC_ISSUER` en `localhost:8000`, así que en AWS debes poner el issuer con tu dominio/IP pública.

Ejemplo (por ahora con IP; mejor con dominio + HTTPS):

```
export OIDC_ISSUER="http://<EC2_PUBLIC_IP>:8000"
export CLIENT_ID="tfg-client"
export CLIENT_SECRET="tfg-secret"
export JWT_SECRET="CHANGE_ME_TO_A_STRONG_SECRET"
export JWT_ALG="HS256"
```

Ideal: **no hardcodear secretos**, siempre env vars.

5) Arranque automático con systemd (recomendado)

Crea el servicio:

```
sudo nano /etc/systemd/system/fake-oidc.service
```

Pega (ajusta `User`, paths y `OIDC_ISSUER`):

```
[Unit]
Description=Fake OIDC Provider (FastAPI)
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/fake_oidc
Environment=OIDC_ISSUER=http://<EC2_PUBLIC_IP>:8000
Environment=CLIENT_ID=tfg-client
Environment=CLIENT_SECRET=tf-g-secret
Environment=JWT_SECRET=CHANGE_ME_TO_A_STRONG_SECRET
Environment=JWT_ALG=HS256
ExecStart=/home/ubuntu/fake_oidc/.venv/bin/uvicorn main:app --host
0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

Activa y arranca:

```
sudo systemctl daemon-reload
sudo systemctl enable fake-oidc
sudo systemctl start fake-oidc
sudo systemctl status fake-oidc
```

Ya debería responder en:

- `http://<EC2_PUBLIC_IP>:8000`

6) (Opcional pero recomendado) Nginx + HTTPS gratis

Esto te evita exponer el puerto 8000 y te da HTTPS.

Instala Nginx

```
sudo apt install -y nginx
```

Config Nginx:

```
sudo nano /etc/nginx/sites-available/fake-oidc
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Activa:

```
sudo ln -s /etc/nginx/sites-available/fake-oidc /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

HTTPS con Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d your-domain.com
```

Luego tu issuer debería ser:

- `https://your-domain.com`

Qué NO usar si quieres “free-tier máximo”

- **ALB/Load Balancer:** cobra por hora y por LCU.
- Elastic Beanstalk puede servir, pero si te crea un LB te rompe el objetivo “casi gratis”. EB como servicio no cobra, pero pagas recursos subyacentes.

Dos detalles importantes (para evitar dolores luego)

1. **IP pública cambia** si paras/enciendes la instancia (a menos que uses Elastic IP; Elastic IP puede costar si no está asociada/activa).
 2. Si en tu Spring usas `issuer-uri`, normalmente Spring espera `/.well-known/openid-configuration`. Tu fake OIDC ahora mismo suele necesitar ese endpoint para integrarse "como OIDC real".
-

Si me dices **cómo quieres exponer el servidor**:

- **A) con IP pública** (rápido)
- **B) con dominio + HTTPS** (lo más realista)

te dejo los comandos exactos ya personalizados (issuer final, puertos a abrir/cerrar y config final del servicio).

You said:

el fake-oidc lo puedo lanzar en el ordenador en local?

ChatGPT said:

Sí, totalmente. De hecho es **lo mejor** para empezar: primero lo ejecutas en local, lo pruebas bien, y cuando todo funciona, lo subes tal cual a AWS.

Con tu fake-OIDC puedes hacerlo así:

1. Entra en la carpeta del proyecto:
`cd fake_oidc`
2. Crea y activa un entorno virtual:
`python -m venv .venv`
`source .venv/bin/activate` *# Linux / macOS*
`# o`
`.venv\Scripts\activate` *# Windows*
3. Instala dependencias:
`pip install -r requirements.txt`

4. Define las variables de entorno (muy importante):

En Linux / macOS:

```
export OIDC_ISSUER="http://localhost:8000"
export CLIENT_ID="tfg-client"
export CLIENT_SECRET="tfg-secret"
export JWT_SECRET="super-secret"
export JWT_ALG="HS256"
```

En Windows (PowerShell):

```
$env:OIDC_ISSUER="http://localhost:8000"
$env:CLIENT_ID="tfg-client"
$env:CLIENT_SECRET="tfg-secret"
$env:JWT_SECRET="super-secret"
$env:JWT_ALG="HS256"
```

5. Lanza el servidor:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

6. Comprueba que responde:

Abre el navegador o usa curl:

- <http://localhost:8000/authorize>
- <http://localhost:8000/token>
- <http://localhost:8000/userinfo>

Si ves respuestas (aunque sean errores por falta de parámetros), el servidor está vivo.

Además, FastAPI te da documentación automática:

```
http://localhost:8000/docs
```

Ahí puedes probar todos los endpoints cómodamente.

Este flujo local es exactamente el mismo que luego usarás en AWS.
La única diferencia al desplegar será:

- Cambiar `OIDC_ISSUER` a la URL pública de tu EC2 o dominio.
- Cambiar `--host` a `0.0.0.0` (que ya lo estás usando).
- Posiblemente añadir `systemd` o `Docker`.

Así que sí: tu fake-OIDC está pensado perfectamente para lanzarse primero en local.

You said:
no tengo python instalado, que version me debería instalar

ChatGPT said:

Instala **Python 3.11 o 3.12**.
Cualquiera de las dos es perfecta para tu fake-OIDC.

Si quieres una recomendación concreta: **Python 3.11**.
Es la más estable ahora mismo y todas las librerías que usas (FastAPI, uvicorn, python-jose, etc.) están totalmente probadas ahí.

Tu requirements.txt no tiene nada que exija versiones raras, así que 3.11 va a ir fino.

Resumen rápido:

Versión	¿Vale?	Motivo
Python 3.10		Ya empieza a quedarse atrás
Python 3.11	 Recomendado	Estable, rápida, compatible con todo
Python 3.12		También válida, más nueva
Python 3.13		Demasiado nueva, posibles incompatibilidades

Según tu sistema:

Windows:

Descarga de

<https://www.python.org/downloads/>

Marca esta opción al instalar:

☒ Add Python to PATH

Luego comprueba:

```
python --version
```

o en Windows a veces:

```
py --version
```

Deberías ver algo como:

```
Python 3.11.x
```

Con eso ya puedes seguir exactamente los pasos para levantar tu fake-OIDC en local.

You said:

para que sirve cada uno de estas variables

```
$env:OIDC_ISSUER="http://localhost:8000" $env:CLIENT_ID="tfg-client"
```

```
$env:CLIENT_SECRET="tfg-secret" $env:JWT_SECRET="super-secret"
```

```
$env:JWT_ALG="HS256"
```

ChatGPT said:

Estas variables son el "corazón" de tu fake-OIDC. Definen quién eres como proveedor de identidad y cómo se firman/validan los tokens.

Vamos una por una:

```
$env:OIDC_ISSUER="http://localhost:8000"
```

Es la identidad de tu servidor OIDC.
En los tokens que generes aparecerá así:

```
{  
  "iss": "http://localhost:8000"  
}
```

Y cuando Spring valide un token, comprobará:

“¿El `iss` del token coincide exactamente con el issuer que yo espero?”

Si no coincide → token inválido.

Por eso:

- En local: `http://localhost:8000`
- En AWS: `https://fake-oidc.tudominio.com`

```
$env:CLIENT_ID="tfg-client"
```

Es el identificador de tu aplicación cliente.
Representa a tu Spring Boot / frontend que “confía” en este OIDC.

En OAuth2 significa:

“Esta app está registrada en el proveedor con este ID”

Cuando tu app haga `/authorize` o `/token`, enviará ese `client_id`.

```
$env:CLIENT_SECRET="tfg-secret"
```

Es la contraseña de tu aplicación cliente.

En el flujo:

1. El cliente manda `client_id`

2. El servidor le pide prueba de identidad → `client_secret`
3. Si coincide → puede obtener tokens

En producción esto sería **secreto absoluto**.

Aquí es un fake, pero conceptualmente es igual que en Cognito/Keycloak.

```
$env:JWT_SECRET="super-secret"
```

Es la clave con la que firmas los JWT.

Tu fake-OIDC genera tokens como:

```
HEADER.payload.SIGNATURE
```

La `SIGNATURE` se calcula con:

```
HMAC(JWT_SECRET, header + payload)
```

Y Spring usará el **mismo secret** para validar que el token no fue manipulado.

Si cambias este valor:

- Todos los tokens anteriores dejan de ser válidos.

```
$env:JWT_ALG="HS256"
```

Es el algoritmo de firma del JWT.

`HS256` significa:

HMAC + SHA-256 + clave compartida (JWT_SECRET)

Otros ejemplos:

- RS256 → clave privada + clave pública (más "real OIDC")
- HS256 → más simple para un fake

Para tu proyecto, HS256 es perfecto.

En conjunto

Estas 5 variables hacen que tu fake-OIDC funcione como un proveedor real:

Variable	Qué representa
<code>OIDC_ISSUER</code>	“Quién soy” como proveedor
<code>CLIENT_ID</code>	“Quién es la aplicación cliente”
<code>CLIENT_SECRET</code>	“Cómo se autentica la aplicación”
<code>JWT_SECRET</code>	Clave para firmar tokens
<code>JWT_ALG</code>	Algoritmo de firma

Si lo llevas a producción real, el esquema es el mismo:
solo cambian los valores, no el concepto.

Ahora mismo tu fake-OIDC está imitando exactamente:

lo que hace Cognito, Keycloak, Auth0, etc., pero en versión mínima.

You said:

cual es la configuracion de spring para poder usar el servidor

ChatGPT said:

Depende de *cómo* vas a usar el fake-OIDC en Spring. En tu caso tienes dos opciones típicas:

1. Tu Spring Boot es un **OAuth2 Client** (login con OIDC)

2. Tu Spring Boot es un **Resource Server** (valida JWT que vienen del fake-OIDC)

Por lo que has contado, ahora mismo quieres validar tokens → eres un **Resource Server**.

Como usas HS256 y una secret compartida (JWT_SECRET), NO puedes usar issuer-uri directamente (Spring ahí espera discovery + jwks).

Tienes que configurar el decoder manualmente con la misma secret.

application.yml:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          # No usamos issuer-uri aquí
          # Configuramos el decoder manualmente en Java
```

Y creas un bean en Spring:

```
package com.yourorg.library.security;

import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.jwt.*;

@Configuration
public class JwtConfig {

    private static final String SECRET = "super-secret"; // mismo valor
    // que JWT_SECRET
    private static final String ALGORITHM = "HmacSHA256";
    private static final String ISSUER = "http://localhost:8000"; //
    // mismo que OIDC_ISSUER

    @Bean
    public JwtDecoder jwtDecoder() {
        SecretKeySpec secretKey = new SecretKeySpec(
            SECRET.getBytes(StandardCharsets.UTF_8),
            ALGORITHM
        );
    }
```

```

        NimbusJwtDecoder decoder = NimbusJwtDecoder
            .withSecretKey(secretKey)
            .macAlgorithm(MacAlgorithm.HS256)
            .build();

        // Validar el issuer también
        OAuth2TokenValidator<Jwt> issuerValidator =
            JwtValidators.createDefaultWithIssuer(ISSUER);

        decoder.setJwtValidator(issuerValidator);

        return decoder;
    }
}

```

Y tu `SecurityConfig` queda así:

```

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/admin/**").hasRole("LIBRARIAN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 ->
                oauth2.jwt(jwt -> {}))
        );

        return http.build();
    }
}

```

Con esto Spring hará:

1. Recibe `Authorization: Bearer <token>`
2. Usa la secret `"super-secret"`
3. Verifica:
 - firma HS256
 - expiración

- issuer == http://localhost:8000
4. Si todo está bien → request autenticada

Si luego más adelante quieres que tu fake-OIDC sea "100% OIDC realista", tendrás que:

- Cambiar a RS256
- Exponer:
 - /.well-known/openid-configuration
 - /jwks.json
- Y entonces sí podrás usar:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8000
```

Pero para ahora, con HS256, esta configuración manual es la correcta y la más simple para tu MVP.

You said:

en spring boot como coger un valor del application config

ChatGPT said:

En Spring Boot tienes varias formas de leer valores del application.yml o application.properties. Las más usadas son estas:

Supongamos que en tu application.yml tienes:

```
security:
  jwt:
    secret: super-secret
    issuer: http://localhost:8000
```

o en application.properties:

```
security.jwt.secret=super-secret  
security.jwt.issuer=http://localhost:8000
```

1. Con @Value (rápido y directo)

```
@Component  
public class JwtProperties {  
  
    @Value("${security.jwt.secret}")  
    private String secret;  
  
    @Value("${security.jwt.issuer}")  
    private String issuer;  
  
    public String getSecret() {  
        return secret;  
    }  
  
    public String getIssuer() {  
        return issuer;  
    }  
}
```

Uso:

```
@Service  
public class TokenService {  
  
    private final JwtProperties jwtProperties;  
  
    public TokenService(JwtProperties jwtProperties) {  
        this.jwtProperties = jwtProperties;  
    }  
  
    public void example() {  
        System.out.println(jwtProperties.getSecret());  
        System.out.println(jwtProperties.getIssuer());  
    }  
}
```

Es simple, pero no es la forma más "limpia" para muchos valores.

2. Con `@ConfigurationProperties` (la forma profesional y escalable)

Es la más recomendable cuando tienes varias propiedades relacionadas.

```
package com.yourorg.library.security;

import
org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Component
@ConfigurationProperties(prefix = "security.jwt")
public class JwtProperties {

    private String secret;
    private String issuer;

    public String getSecret() {
        return secret;
    }

    public void setSecret(String secret) {
        this.secret = secret;
    }

    public String getIssuer() {
        return issuer;
    }

    public void setIssuer(String issuer) {
        this.issuer = issuer;
    }
}
```

Y tu `application.yml`:

```
security:
  jwt:
    secret: super-secret
    issuer: http://localhost:8000
```

Uso:

```

@Configuration
public class JwtConfig {

    private final JwtProperties properties;

    public JwtConfig(JwtProperties properties) {
        this.properties = properties;
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        String secret = properties.getSecret();
        String issuer = properties.getIssuer();
        ...
    }
}

```

Esta es la que más se parece a cómo trabajan los proyectos reales.

3. Usar directamente variables de entorno

Spring las mapea automáticamente:

```

export SECURITY_JWT_SECRET=super-secret
export SECURITY_JWT_ISSUER=http://localhost:8000

```

Y Spring las verá como:

```

security:
  jwt:
    secret: ${SECURITY_JWT_SECRET}
    issuer: ${SECURITY_JWT_ISSUER}

```

Esto es ideal cuando despliegas en AWS:

- No pones secretos en el repo
 - Todo viene del entorno
-

4. Acceso directo desde `Environment` (menos usado)

```
@Autowired
private Environment env;

String secret = env.getProperty("security.jwt.secret");
```

Útil para casos muy puntuales, pero peor diseño que `@ConfigurationProperties`.

Para tu proyecto concreto, la combinación ideal es:

- En local:

```
security:
  jwt:
    secret: super-secret
    issuer: http://localhost:8000
```

- En AWS:

```
export SECURITY_JWT_SECRET=...
export SECURITY_JWT_ISSUER=https://fake-oidc.tudominio.com
```

y sigues usando el mismo `JwtProperties` sin cambiar código.

Eso es exactamente cómo se hace en proyectos profesionales.

You said:

no, quiero usarlo como OAuth2 client

ChatGPT said:

Thought for 10s

Perfecto — si lo quieres usar como **OAuth2 Client (Login con OIDC)** en Spring Boot, la configuración es distinta a "resource server".

Tu fake-OIDC ya tiene estos endpoints:

- `/authorize`
- `/token`
- `/userinfo`

Eso **encaja** con OAuth2/OIDC login. El único "pero" típico es que Spring, para OIDC, **valida la firma del** `id_token` y normalmente espera **JWKS (RS256)**. Como tu fake usa **HS256 + secret compartida**, hay que decirle a Spring cómo validar el `id_token` con esa secret.

Abajo tienes una configuración **funcional** (en inglés el código/config, como quieres).

1) application.yml (OAuth2 Client)

```
security:
  oidc:
    issuer: "http://localhost:8000"
    jwt-secret: "super-secret" # same as fake OIDC JWT_SECRET

spring:
  security:
    oauth2:
      client:
        registration:
          fakeoidc:
            client-id: "tfg-client"
            client-secret: "tfg-secret"
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope:
              - openid
              - profile
              - email
        provider:
          fakeoidc:
            authorization-uri: "http://localhost:8000/authorize"
            token-uri: "http://localhost:8000/token"
            user-info-uri: "http://localhost:8000/userinfo"
            user-name-attribute: "sub"
            # issuer-uri would normally be used with discovery:
            # issuer-uri: "http://localhost:8000"
```

Ojo: Aquí **NO** uso `issuer-uri` porque tu fake no expone `/.well-known/openid-configuration`. Si en el futuro lo implementas, puedes simplificar.

2) SecurityConfig (habilitar oauth2Login)

```
package com.yourorg.library.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login/**", "/error").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> {});

        return http.build();
    }
}
```

Con esto:

- Si entras a una ruta protegida, Spring te manda al login OIDC automáticamente.
- El callback será: `/login/oauth2/code/fakeoidc`

3) La parte clave: validar `id_token` HS256 (custom JwtDecoderFactory)

Spring Security para OIDC login valida el `id_token`.

Como tu fake firma con **HS256**, debes darle a Spring un decoder basado en **secret compartida**.

```
package com.yourorg.library.security;

import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.oauth2.client.registration.ClientRegistration;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.security.oauth2.client.authentication.JwtDecoderFactory;
import org.springframework.security.oauth2.core.DelegatingOAuth2TokenValidator;
import org.springframework.security.oauth2.core.OAuth2TokenValidator;

@Configuration
public class OidcHs256IdTokenConfig {

    @Value("${security.oidc.jwt-secret}")
    private String jwtSecret;

    @Value("${security.oidc.issuer}")
    private String issuer;

    @Bean
    public JwtDecoderFactory<ClientRegistration> idTokenDecoderFactory()
    {
        return (ClientRegistration registration) -> {
            SecretKeySpec key = new SecretKeySpec(
                jwtSecret.getBytes(StandardCharsets.UTF_8),
                "HmacSHA256"
            );

            NimbusJwtDecoder decoder = NimbusJwtDecoder
                .withSecretKey(key)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
        };
    }
}
```

```

        OAuth2TokenValidator<Jwt> withIssuer =
JwtValidators.createDefaultWithIssuer(issuer);
        // If your fake sets "aud" to client_id, this will also help:
        OAuth2TokenValidator<Jwt> withAudience =
            new JwtClaimValidator<>("aud", aud -> aud != null &&
aud.toString().contains(registration.getClientId()));

        decoder.setJwtValidator(new
DelegatingOAuth2TokenValidator<>(withIssuer, withAudience));
        return decoder;
    };
}
}

```

✓ Con esto, Spring podrá:

- intercambiar code por tokens en /token
- recibir id_token firmado con HS256
- **validarlo con tu** JWT_SECRET
- llamar a /userinfo para completar el usuario

4) Cómo probarlo en local (flujo real)

1. Arranca el fake OIDC en `http://localhost:8000`
2. Arranca tu Spring Boot
3. Visita cualquier endpoint protegido, por ejemplo:
 - `http://localhost:8080/api/whatever`
4. Spring te redirige a `http://localhost:8000/authorize?...`
5. Al volver al callback, Spring termina el login

Importante: requisitos mínimos que debe cumplir tu fake para que esto funcione

Para `oauth2Login`, tu fake debe:

- en `/authorize`, devolver un `code` válido y redirigir al `redirect_uri`
- en `/token`, devolver al menos:
 - `access_token`
 - `id_token` (para OIDC)
 - `token_type`
 - `expires_in`
- en `/userinfo`, devolver `sub` (y lo que quieras: email, name...)

Si tu `/token` no devuelve `id_token`, Spring lo tratará como OAuth2 (no OIDC) y no tendrás "OIDC user".

Si quieres hacerlo "más estándar OIDC" (opcional)

Lo normal en OIDC es firmar con **RS256** y exponer:

- `/.well-known/openid-configuration`
- `jwks_uri` (por ejemplo `/jwks.json`)

Ahí sí podrías configurar solo:

```
spring.security.oauth2.client.provider.fakeoidc.issuer-uri:  
"http://localhost:8000"
```

y Spring lo descubre todo.

Si me pegas **la respuesta JSON actual de tu** `/token` (solo un ejemplo, sin secretos), te ajusto la config al 100% para tu fake: claims (`iss`, `aud`, `sub`), scopes y `user-name-attribute`.

You said:

mi fake_oidc no puede usar JWKS (RS256)?

ChatGPT said:

Sí, **puede**. No hay nada "intrínseco" en tu fake-OIDC que lo impida: ahora mismo estás firmando JWT con **HS256** (secreto compartido). Puedes cambiarlo para usar **RS256 + JWKS**, que es lo estándar en OIDC y además te simplifica muchísimo la configuración en Spring (porque Spring puede usar `issuer-uri` y descubrir todo automáticamente).

Lo que cambia es:

- En **HS256**: servidor y cliente comparten `JWT_SECRET` (menos realista y menos compatible con OIDC estándar).
- En **RS256**: el servidor firma con **clave privada**, y publica la **clave pública** en **JWKS** (`jwks_uri`). El cliente solo necesita leer la pública.

Qué necesitas añadir a tu fake-OIDC para "RS256 + JWKS"

1. Generar un par de claves RSA (private/public)
2. Publicar un endpoint `GET /jwks.json` (o similar)
3. (Recomendado) Publicar `GET /.well-known/openid-configuration`
4. Firmar `id_token` (y si quieres `access_token`) con RS256 usando la privada
5. Incluir un `kid` en el header del JWT que coincida con el JWKS

Implementación mínima (FastAPI) — en inglés

1) Instala dependencias

La forma más común es `cryptography` + `python-jose`:

```
pip install cryptography python-jose
```

2) Genera claves RSA (una vez)

Puedes generarlas con OpenSSL:

```
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem
```

Guárdalas en tu servidor (en AWS: idealmente con permisos restrictivos o variables/secret manager).

3) Carga claves y expón JWKS

Ejemplo (muy directo) para `jwks.json`:

```
from jose.utils import base64url_encode
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.backends import default_backend
import json

KID = "fake-oidc-key-1"

def load_private_key_pem(path: str):
    with open(path, "rb") as f:
        return serialization.load_pem_private_key(f.read(),
password=None, backend=default_backend())

def load_public_key_pem(path: str):
    with open(path, "rb") as f:
        return serialization.load_pem_public_key(f.read(),
backend=default_backend())

private_key = load_private_key_pem("private.pem")
public_key = load_public_key_pem("public.pem")

def public_key_to_jwk(pub_key) -> dict:
    numbers = pub_key.public_numbers()
    n = base64url_encode(numbers.n.to_bytes((numbers.n.bit_length() + 7)
// 8, "big")).decode()
    e = base64url_encode(numbers.e.to_bytes((numbers.e.bit_length() + 7)
// 8, "big")).decode()

    return {
        "kty": "RSA",
        "use": "sig",
        "alg": "RS256",
        "kid": KID,
        "n": n,
        "e": e
    }

JWKS = {"keys": [public_key_to_jwk(public_key)]}
```

Y endpoint:

```

from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/jwks.json")
def jwks():
    return JSONResponse(JWKS)

```

4) Firmar tokens con RS256 + kid

Cuando generes el `id_token`:

```

from jose import jwt
import time

ISSUER = "http://localhost:8000" # your OIDC_ISSUER
CLIENT_ID = "tfg-client"

def create_id_token(subject: str) -> str:
    now = int(time.time())
    claims = {
        "iss": ISSUER,
        "sub": subject,
        "aud": CLIENT_ID,
        "iat": now,
        "exp": now + 3600,
    }

    token = jwt.encode(
        claims,
        private_key,
        algorithm="RS256",
        headers={"kid": KID}
    )
    return token

```

Con eso, Spring puede validar el `id_token` leyendo tu `jwks.json`.

(Muy recomendado) Endpoint discovery

Spring "OIDC standard" espera:

```
GET /.well-known/openid-configuration
```

Ejemplo mínimo:

```
@app.get("/.well-known/openid-configuration")
def openid_configuration():
    issuer = ISSUER
    return {
        "issuer": issuer,
        "authorization_endpoint": f"{issuer}/authorize",
        "token_endpoint": f"{issuer}/token",
        "userinfo_endpoint": f"{issuer}/userinfo",
        "jwks_uri": f"{issuer}/jwks.json",
        "response_types_supported": ["code"],
        "subject_types_supported": ["public"],
        "id_token_signing_alg_values_supported": ["RS256"],
        "scopes_supported": ["openid", "profile", "email"]
    }
```

Ventaja inmediata en Spring Boot (OAuth2 Client)

Cuando tengas discovery + jwks, tu Spring se vuelve súper simple:

```
spring:
  security:
    oauth2:
      client:
        registration:
          fakeoidc:
            client-id: tfg-client
            client-secret: tfg-secret
            scope: [openid, profile, email]
        provider:
          fakeoidc:
            issuer-uri: http://localhost:8000
```

Y listo: Spring descubre authorization/token/userinfo/jwks automáticamente.

¿Entonces por qué no usar RS256 desde el principio?

Puedes, y de hecho es mejor para:

- compatibilidad con Spring OIDC Login
- que sea "realista" (como Cognito/Keycloak/Auth0)
- no compartir secretos entre servidor y cliente

Único coste: implementar `jwks.json` y firmar con clave privada (lo de arriba).

Si quieres, te adapto tu `main.py` actual exactamente (sin inventar rutas nuevas) para:

- RS256
- `/jwks.json`
- `/.well-known/openid-configuration`
- y que `/token` devuelva un `id_token` válido para Spring

Solo dime: ¿tu `fake_oidc` ahora mismo está devolviendo `id_token` en `/token` o solo `access_token`?

You said:

Cognito puede funcionar como servidor para OAuth?

ChatGPT said:

Sí. Amazon Cognito es, de hecho, un **servidor OAuth2 y OpenID Connect completo**. Funciona exactamente como tu fake-OIDC, pero "de verdad" y gestionado por AWS.

Cognito implementa:

- OAuth2 Authorization Server
- OpenID Connect Provider
- JWT firmados con RS256

- JWKS público
- Discovery endpoint
- Gestión de usuarios, contraseñas, MFA, etc.

Es decir, Cognito puede reemplazar a tu fake-OIDC cuando quieras pasar a algo real.

Con Cognito tienes:

Tu fake OIDC	Cognito
/authorize	/oauth2/authorize
/token	/oauth2/token
/userinfo	/oauth2/userInfo
/jwks.json	/.well-known/jwks.json
/.well-known/openid-configuration	Incluido automáticamente
HS256	RS256 estándar
Claves manuales	Claves gestionadas por AWS

Ejemplo de issuer real de Cognito:

https://cognito-idp.eu-west-1.amazonaws.com/eu-west-1_XXXXXXX

Ahí:

- iss del token = ese valor
 - Spring lo usa para descubrir todo automáticamente
-

En Spring Boot (OAuth2 Client) con Cognito la configuración es limpia:

```
spring:
  security:
    oauth2:
      client:
        registration:
          cognito:
            client-id: YOUR_CLIENT_ID
            client-secret: YOUR_CLIENT_SECRET
            scope:
              - openid
              - email
              - profile
        provider:
          cognito:
            issuer-uri: https://cognito-idp.eu-west-1.amazonaws.com/eu-west-1_XXXXXXX
```

Y ya está. Spring:

1. Descubre endpoints
2. Descarga JWKS
3. Valida firmas RS256
4. Maneja login OIDC completo

Sin código extra.

Para tu proyecto, el uso lógico es:

1. Ahora:
 - Usas tu **fake-OIDC** para aprender, probar y desarrollar.
2. Más adelante:
 - Cambias a **Cognito** sin tocar tu lógica de seguridad de Spring, solo cambiando:
 - issuer-uri
 - client-id
 - client-secret

Ese es exactamente el flujo profesional real.

Sobre el Free Tier:

Cognito tiene un **free tier bastante generoso**:

- Hasta 50.000 usuarios activos mensuales (en User Pools) durante el primer año.
- No necesitas EC2 ni servidores propios.
- No pagas por el "servidor OAuth", solo por uso real.

Pero para aprendizaje puro, tu fake-OIDC es perfecto porque:

- entiendes qué hace cada endpoint
- sabes qué está pasando por debajo
- luego Cognito te resulta trivial.

You said:

Voy a usar amazon cognito en vez de fake-oidc. Estoy configurando amazon cognito. Me está preguntando que tipo de aplicacion voy a hacer "traditional web application - an application hosted on a webserver. Uses redirects and separate pages to display info" o Machine to machine application "platform independent server to serves authorizes api access with ouath

ChatGPT said:

En tu caso tienes que elegir **Traditional web application**.

Porque tu arquitectura es esta:

- Un usuario real entra desde un navegador.
- Tu aplicación Spring Boot:
 - redirige al usuario a Cognito para que haga login,
 - Cognito devuelve al usuario a tu aplicación con un code,
 - Spring intercambia ese code por tokens (id_token, access_token),
 - el usuario queda autenticado en tu web.

Eso es exactamente el flujo **Authorization Code Flow**, que es lo que Cognito llama *Traditional web application*.

La opción **Machine to machine application** es otra cosa totalmente distinta:

- No hay usuario humano.
- No hay navegador.
- No hay redirecciones.
- Es para servicios backend que se autentican entre ellos usando `client_id` + `client_secret` (Client Credentials Flow).
- Ejemplo: un microservicio llamando a otro microservicio.

Tú no quieres eso ahora.

Resumiendo:

Opción de Cognito	Para qué sirve	¿Es tu caso?
Traditional web application	Login de usuarios con navegador y redirecciones	☒ Sí
Machine to machine	Servicios que se autentican entre sí sin usuarios	☐ No

Cuando elijas **Traditional web application**, Cognito va a crear:

- Un *App client* con:
 - `client_id`
 - `client_secret`
- Habilitará:
 - OAuth2 Authorization Code
 - OpenID Connect
- Te pedirá:
 - Redirect URLs (por ejemplo: `http://localhost:8080/login/oauth2/code/cognito`)
 - Logout URLs (opcional)

Que encaja exactamente con cómo funciona Spring Boot con `oauth2Login()`.

Estás haciendo lo correcto pasando de fake-OIDC a Cognito:
tu sistema pasa de "simulación" a "proveedor OIDC real" sin cambiar arquitectura.

You said:

Add a return URL - optional Choose a return URL. Cognito redirects to this URL after successful sign-in with the managed login pages on your user pool domain. Your application can then process the resulting tokens. Return URL Info https:// The length of the return URL must be between 1 and 1024 characters. Valid characters are letters, marks, numbers, symbols, and punctuations. Amazon Cognito requires HTTPS over HTTP except for http://localhost for testing purposes only. App return URLs such as myapp://example are also supported. Must not contain a fragment. Tengo que rellenar esto?

ChatGPT said:

Sí, tienes que rellenarlo.

Ese campo es el **redirect URI** de tu aplicación, es decir, la URL a la que Cognito va a devolver al usuario después de hacer login.

Si no lo pones, Cognito no sabe "a dónde volver" tras autenticarse y el flujo OAuth no funciona.

En Spring Boot con `oauth2Login`, el callback por defecto es:

```
/login/oauth2/code/{registrationId}
```

Así que, si tu aplicación corre en local en el puerto 8080 y tu registro se llama por ejemplo `cognito`, entonces debes poner:

```
http://localhost:8080/login/oauth2/code/cognito
```

Ese es el valor correcto para desarrollo.

Ejemplo completo:

Return URL:

```
http://localhost:8080/login/oauth2/code/cognito
```

Más adelante, cuando despliegues tu aplicación en producción, añadirás otra:

```
https://miapp.midominio.com/login/oauth2/code/cognito
```

Cognito permite tener **varias Return URLs** en el mismo App Client, así que puedes tener:

- una para local
 - otra para producción
 - otra para pruebas
-

Por qué es tan importante:

El flujo real es:

1. Spring redirige al navegador a Cognito:
`/oauth2/authorize?...&redirect_uri=http://localhost:8080/login/oauth2/code/cognito`
2. El usuario hace login en Cognito.
3. Cognito redirige a:
`http://localhost:8080/login/oauth2/code/cognito?code=XXXX`
4. Spring recibe el `code`, llama al `/token` de Cognito y obtiene:
 - `id_token`
 - `access_token`

Si la Return URL que pones en Cognito **no coincide exactamente** con la que usa Spring:

- Cognito rechazará la petición
 - Te dará error de "redirect_uri mismatch"
-

Así que ahora mismo, pon:

```
http://localhost:8080/login/oauth2/code/cognito
```

Es exactamente lo que Spring espera por defecto y te ahorra configuración extra.

You said:

ahora quiero la configuracion para spring boot

ChatGPT said:

Aquí tienes una configuración **lista** para usar **Amazon Cognito como OAuth2/OIDC Client** (login) en **Spring Boot**. Todo el código en inglés.

1) Dependencias (Gradle)

Asegúrate de tener estas:

```
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-web")  
    implementation("org.springframework.boot:spring-boot-starter-security")  
    implementation("org.springframework.boot:spring-boot-starter-oauth2-  
client")  
}
```

Si ya tienes `spring-boot-starter-oauth2-resource-server`, puedes mantenerlo, pero para *login* lo importante es `oauth2-client`.

2) application.yml (Cognito OIDC Login)

Pon tu `client-id`, `client-secret` y el `issuer-uri` del User Pool.

```
server:  
  port: 8080  
  
spring:  
  security:  
    oauth2:  
      client:  
        registration:  
          cognito:  
            client-id: YOUR_COGNITO_CLIENT_ID
```



```

    client-secret: YOUR_COGNITO_CLIENT_SECRET
    scope:
      - openid
      - email
      - profile
    authorization-grant-type: authorization_code
    redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"

    provider:
      cognito:
        issuer-uri: https://cognito-
idp.YOUR_REGION.amazonaws.com/YOUR_USER_POOL_ID

```

¿De dónde sacas el issuer-uri?

Tiene esta forma:

```
https://cognito-idp.eu-west-1.amazonaws.com/eu-west-1_XXXXXXXXXX
```

- YOUR_REGION: la región (ej. eu-west-1)
- YOUR_USER_POOL_ID: el ID del user pool (ej. eu-west-1_AbCdEf123)

Con issuer-uri, Spring descubre solo:

- authorize endpoint
- token endpoint
- userinfo
- jwks

3) SecurityConfig (habilitar oauth2Login)

```

package com.yourorg.library.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean

```

```

SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/public/**", "/error").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(oauth2 -> {});

    return http.build();
}
}

```

Con esto:

- cualquier endpoint (excepto / y /public/**) fuerza login en Cognito.

4) Probar rápido: endpoint que devuelve el usuario logueado

```

package com.yourorg.library.controller;

import java.util.Map;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.OidcUser;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class MeController {

    @GetMapping("/me")
    public Map<String, Object> me(@AuthenticationPrincipal OidcUser user)
    {
        return Map.of(
            "subject", user.getSubject(),
            "email", user.getEmail(),
            "claims", user.getClaims()
        );
    }
}

```

```
}  
}
```

Prueba:

- abre `http://localhost:8080/api/me`
- te redirige a Cognito
- tras login, te devuelve claims

5) Configuración Cognito que debe coincidir

En Cognito (App client):

Return URL (callback)

Debe contener exactamente:

```
http://localhost:8080/login/oauth2/code/cognito
```

("cognito" es el `registrationId` de tu `application.yml`)

Logout URL (opcional pero recomendable)

Puedes añadir:

```
http://localhost:8080/
```

6) (Opcional) Logout que cierre sesión en Cognito también

Por defecto Spring solo "olvida" la sesión local. Para logout completo, normalmente rediriges al logout de Cognito.

Añade en `application.yml`:

```
app:  
  cognito:  
    domain: https://YOUR_DOMAIN_PREFIX.auth.YOUR_REGION.amazoncognito.com
```

Y en `SecurityConfig`:

```
package com.yourorg.library.security;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Value("${app.cognito.domain}")
    private String cognitoDomain;

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/public/**", "/error").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> {})
            .logout(logout -> logout
                .logoutSuccessUrl(cognitoDomain +
"/logout?client_id=YOUR_COGNITO_CLIENT_ID&logout_uri=http://localhost:8080/")
            );

        return http.build();
    }
}
```

(Para producción, cambias `logout_uri` por tu dominio HTTPS.)

Si me dices 3 datos, te la dejo 100% pegable

1. Región (`eu-west-1`?)

2. User Pool ID (eu-west-1_...)
3. Domain prefix (xxxx.auth.eu-west-1.amazoncognito.com)

y te devuelvo el `application.yml` final sin "placeholders".

You said:

hago bien el login pero me da invalid credentials. Creo que es porque no encuentra el localhost para enviar la confirmacion del login

ChatGPT said:

Thought for 30s

No es porque "Cognito no encuentre localhost". Cognito **no llama a tu localhost desde AWS**. En el flujo OAuth2/OIDC, **quien vuelve a tu app es el navegador del usuario** mediante una redirección. AWS incluso permite `http://localhost` como callback solo para pruebas.

Si te aparece **"Invalid credentials"** después de que el login en Cognito parezca correcto, casi siempre es porque **Spring falla al canjear el code por tokens** o porque **no puede validar el token** (issuer/JWKS), o porque hay un problema con **state/session**.

Aquí tienes los 6 checks que suelen arreglarlo (en orden):

1) Asegura que el Callback URL en Cognito coincide EXACTAMENTE

En Cognito (App client → Callback / Return URL) debe estar:

`http://localhost:8080/login/oauth2/code/cognito`

- Sin / al final
- Mismo puerto
- Mismo "registrationId" (cognito)

Si tu `registrationId` en Spring no se llama `cognito`, cambia la URL en Cognito o el id en Spring.

2) Confirma tu `application.yml` (lo mínimo correcto)

```
spring:
  security:
    oauth2:
      client:
        registration:
          cognito:
            client-id: YOUR_CLIENT_ID
            client-secret: YOUR_CLIENT_SECRET
            scope: [openid, email, profile]
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          cognito:
            issuer-uri: https://cognito-
idp.YOUR_REGION.amazonaws.com/YOUR_USER_POOL_ID
```

Errores típicos:

- Poner como issuer el dominio hosted UI (...auth....amazoncognito.com) ✗
El issuer correcto es el de cognito-
idp.<region>.amazonaws.com/<userPoolId> ✓

3) El "Invalid credentials" suele ser en el `/token` (client secret / auth method)

Cuando Spring recibe el `code`, llama al token endpoint de Cognito. Si el **client secret** está mal (o el app client no lo usa), Cognito responde con error y Spring termina mostrando algo tipo "Invalid credentials".

Comprueba:

- que copiaste el **Client secret** correcto del App Client
- que el App Client tiene habilitado **Authorization code grant**
- que estás usando el tipo **Traditional web application** (confidential client con secret)

4) No mezcles `localhost` y `127.0.0.1`

El login OIDC usa `state` guardado en sesión/cookie. Si entras por:

- `http://127.0.0.1:8080/...` pero el callback va a
- `http://localhost:8080/...`

puede fallar y acabar en error de autenticación.

Usa **siempre** el mismo host (recomendado: `localhost`) para iniciar el flujo.

5) Asegura que tu app está “viva” en el callback

Cuando Cognito redirige a:

`http://localhost:8080/login/oauth2/code/cognito?code=...`

tu Spring debe estar ejecutándose y escuchando en 8080.

Si no está, verías “site can’t be reached”, pero algunas configuraciones acaban en error genérico al reintentar.

6) Mira el log exacto: ahí sale el motivo real

En consola/logs, busca líneas del estilo:

- `invalid_client` → client id/secret mal o método de auth incorrecto
- `redirect_uri_mismatch` → callback URL no coincide
- `Invalid issuer / Failed to validate JWT` → `issuer-uri` incorrecto
- `state parameter mismatch` → problema de sesión/cookies/host

Para sacar más detalle, sube el log de Spring Security:

```
logging:
  level:
    org.springframework.security: DEBUG
```

Lo más probable por lo que describes

Si "el login va bien" y luego cae, lo más común es:

1. **issuer-uri mal** (usando el dominio hosted UI en vez de cognito-idp), o
 2. **client-secret mal** (o app client configurado como public vs confidential),
o
 3. **callback URL no coincide** al 100%.
-

Si pegas aquí **las 5–10 líneas del log** alrededor del error (con `org.springframework.security: DEBUG`), te digo exactamente cuál de las 3 es y la corrección exacta.

You said:

no tengo el client secret

ChatGPT said:

Thought for 19s

Si **no tienes client secret**, no es un problema de "localhost": significa que tu **App Client de Cognito está creado como *public client*** (típico de SPA/móvil). En Cognito, un *public client* **no tiene secreto** y se asegura el flujo usando **PKCE**.

Tienes 2 caminos, según tu tipo de app Spring:

Opción A (recomendada si tu Spring es “web app” en servidor): crear un App Client con secret

Si tu Spring Boot hace el login en servidor (MVC, sesiones, etc.), lo más cómodo es un **confidential client** con `client_secret`.

Qué hacer en Cognito

1. User Pool → **App integration** → **App clients**
2. **Create app client**
3. Elige **Traditional web application** (y asegúrate de activar algo como **Generate client secret**)
4. Habilita **Authorization code grant**
5. Callback URL:
`http://localhost:8080/login/oauth2/code/cognito`

Con esto, Spring Boot queda “normal”, con `client-id` + `client-secret`.

Opción B (si quieres seguir SIN secret): Spring como public client + PKCE

Esto funciona si tu App Client no tiene secret, pero **debes usar PKCE** (Cognito lo soporta).

1) application.yml (sin secret + client-authentication-method none)

```
spring:
  security:
    oauth2:
      client:
        registration:
          cognito:
            client-id: YOUR_CLIENT_ID
            client-authentication-method: none
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            scope: [openid, email, profile]
        provider:
          cognito:
```

```
issuer-uri: https://cognito-  
idp.YOUR_REGION.amazonaws.com/YOUR_USER_POOL_ID
```

Esto marca el cliente como **public** (sin secret). Spring Security documenta este patrón para public clients.

2) SecurityConfig: habilitar PKCE (imprescindible)

En Spring Security, la forma estándar es usar un **AuthorizationRequestResolver** que añada PKCE.

```
package com.yourorg.library.security;  
  
import jakarta.servlet.http.HttpServletRequest;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import  
org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import  
org.springframework.security.oauth2.client.registration.ClientRegistrationRepository;  
import  
org.springframework.security.oauth2.client.web.DefaultOAuth2AuthorizationRequestResolver;  
import  
org.springframework.security.oauth2.client.web.OAuth2AuthorizationRequestResolver;  
import  
org.springframework.security.oauth2.core.endpoint.OAuth2AuthorizationRequest;  
import org.springframework.security.web.SecurityFilterChain;  
  
import java.util.function.Consumer;  
  
@Configuration  
public class SecurityConfig {  
  
    @Bean  
    SecurityFilterChain securityFilterChain(  
        HttpSecurity http,  
        ClientRegistrationRepository clientRegistrationRepository  
    ) throws Exception {  
  
        http  
            .csrf(csrf -> csrf.disable())  
            .authorizeHttpRequests(auth -> auth
```

```

        .requestMatchers("/", "/error").permitAll()
        .anyRequest().authenticated()
    )
    .oauth2Login(oauth2 -> oauth2
        .authorizationEndpoint(authorization -> authorization

.authorizationRequestResolver(pkceResolver(clientRegistrationRepository))
    )
    );

    return http.build();
}

private OAuth2AuthorizationRequestResolver
pkceResolver(ClientRegistrationRepository repo) {
    DefaultOAuth2AuthorizationRequestResolver defaultResolver =
        new DefaultOAuth2AuthorizationRequestResolver(repo,
"/oauth2/authorization");

defaultResolver.setAuthorizationRequestCustomizer(pkceCustomizer());
    return defaultResolver;
}

private Consumer<OAuth2AuthorizationRequest.Builder> pkceCustomizer()
{
    return builder -> {
        // Minimal PKCE parameters (S256)
        // Spring Security will generate these when configured for
        PKCE; this customizer ensures they're present.
        // If your Spring version provides
        OAuth2AuthorizationRequestCustomizers.withPkce(), prefer that.
        String codeVerifier =
org.springframework.security.oauth2.core.endpoint.PkceParameterNames.CODE
_VERIFYIFIER;
        // If you have Spring's built-in withPkce customizer
        available:
        // return
org.springframework.security.oauth2.client.web.OAuth2AuthorizationRequest
Customizers.withPkce();

        // If not available in your version, tell me your Spring
        Security version and I'll give you the exact code.
    };
}
}

```

OJO: En muchas versiones modernas, Spring tiene un helper tipo `OAuth2AuthorizationRequestCustomizers.withPkce()` (como describe Auth0 para Spring Security).

Pero como tú estás en Spring Boot 4.x (muy nuevo), la API exacta puede variar un poco. Si me dices tu versión exacta de `spring-security`, te lo dejo 100% compilable para tu proyecto.

Por qué te da “invalid credentials” ahora

Porque Spring está intentando intercambiar el `code` por tokens en Cognito, y si el cliente es public, **no puede autenticarse con secret**, así que:

- o falla por `invalid_client`
- o falla porque Cognito espera PKCE y tu request no lleva `code_verifier` / `code_challenge`.

PKCE es exactamente para esto.

Qué te recomiendo ahora mismo

Como estás empezando y quieres que funcione sin pelearte:

✅ **Crea un App Client “Traditional web application” con secret (Opción A)** y usa el `client-secret` en Spring.

Si por requisitos del proyecto quieres sí o sí sin secret, hacemos la **Opción B (PKCE)**.

Pásame 2 cosas y te lo dejo exacto:

1. ¿Tu Spring app es **web server con sesiones** (MVC) o es **solo API**?
2. Pega el error real del log (con `org.springframework.security: DEBUG`).
Con eso te digo si estás chocando
con `invalid_client`, `redirect_uri_mismatch` o PKCE.

You said:

quiero desactivar spring security si ejecuto con perfil "local", y que se active con perfil "aws"

ChatGPT said:

Puedes hacerlo muy limpio con **dos** `SecurityFilterChain` **distintos**, cada uno activado por perfil.

La idea:

- Perfil **local** → todo permitido (sin login, sin JWT).
- Perfil **aws** → seguridad normal (Cognito `oauth2Login()` y/o lo que uses).

1) Activa perfiles

Ejecutar local:

```
./gradlew bootRun --args='--spring.profiles.active=local'
```

Ejecutar aws:

```
./gradlew bootRun --args='--spring.profiles.active=aws'
```

Opción recomendada: 2 clases de configuración con `@Profile`

A) `LocalSecurityConfig` (desactiva todo)

```
package com.yourorg.library.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import
```

```

org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@Profile("local")
public class LocalSecurityConfig {

    @Bean
    SecurityFilterChain localSecurityFilterChain(HttpSecurity http)
throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll())
            // Important: disables all authentication mechanisms
            .securityContext(sc -> sc.disable())
            .sessionManagement(sm -> sm.disable());

        return http.build();
    }
}

```

B) AwsSecurityConfig (seguridad activada)

```

package com.yourorg.library.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@Profile("aws")
public class AwsSecurityConfig {

    @Bean
    SecurityFilterChain awsSecurityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/public/**", "/error").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> {});
        // If you also need JWT validation for APIs:

```

```
        // .oauth2ResourceServer(oauth2 -> oauth2.jwt());

        return http.build();
    }
}
```

Con esto, **en local no existe login**, y en aws sí.

2) Config separada por perfil (application-local.yml / application-aws.yml)

src/main/resources/application-local.yml

server:

port: 8080

src/main/resources/application-aws.yml

spring:

security:

oauth2:

client:

registration:

cognito:

client-id: YOUR_CLIENT_ID

client-secret: YOUR_CLIENT_SECRET

scope: [openid, email, profile]

authorization-grant-type: authorization_code

redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"

provider:

cognito:

issuer-uri: https://cognito-

idp.YOUR_REGION.amazonaws.com/YOUR_USER_POOL_ID

Nota importante (para que no “se mezclen” configs)

Asegúrate de que:

- **Solo existe una** SecurityFilterChain activa por perfil.

- No tengas otra clase `SecurityConfig` sin `@Profile`, porque entonces se aplicaría siempre.
-

Si me pegas tu `SecurityConfig` actual (aunque sea corto), te lo adapto exactamente para que no haya conflicto con Boot 4 / Security 6.x.

You said:

tengo el login con amazon cognito, pero quiero añadir el logout

ChatGPT said:

Para que el **logout** funcione "bien" con Cognito tienes 2 cosas:

1. **Cerrar sesión en tu app** (Spring)
2. **Cerrar sesión en Cognito (Hosted UI)** y volver a una URL tuya

Si solo haces logout en Spring, el usuario puede seguir "logueado" en Cognito y al volver a entrar se reautentica sin pedir credenciales.

Abajo tienes la configuración completa.

1) Añade Logout URL en Cognito

En tu **App client** (Cognito → App integration):

- **Allowed sign-out URLs** (Logout / Sign-out URL)

Para local, añade:

```
http://localhost:8080/
```

Y si tienes otra página "post logout", pon esa.

2) application.yml (guarda domain + client-id)

```
app:
  cognito:
    domain: https://YOUR_DOMAIN_PREFIX.auth.YOUR_REGION.amazoncognito.com
    client-id: YOUR_CLIENT_ID
    logout-redirect: http://localhost:8080/
```

domain es el "Hosted UI domain" (el que termina en amazoncognito.com), NO el issuer.

3) SecurityConfig con logout que redirige a Cognito

```
package com.yourorg.library.security;

import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.logout.LogoutSuccessHandler;

@Configuration
public class SecurityConfig {

    @Value("${app.cognito.domain}")
    private String cognitoDomain;

    @Value("${app.cognito.client-id}")
    private String clientId;

    @Value("${app.cognito.logout-redirect}")
    private String logoutRedirect;
```

```

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/public/**", "/error").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(oauth2 -> {})
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessHandler(cognitoLogoutSuccessHandler())
            .invalidateHttpSession(true)
            .clearAuthentication(true)
            .deleteCookies("JSESSIONID")
        );

    return http.build();
}

@Bean
LogoutSuccessHandler cognitoLogoutSuccessHandler() {
    return (request, response, authentication) -> {
        String encoded = URLEncoder.encode(logoutRedirect,
StandardCharsets.UTF_8);
        String url = cognitoDomain
            + "/logout?client_id=" + clientId
            + "&logout_uri=" + encoded;

        response.sendRedirect(url);
    };
}
}

```

Esto hace:

- `/logout` → limpia sesión/cookies de tu app
- redirige al **logout de Cognito**
- Cognito redirige de vuelta a `logoutRedirect`

4) Probar

1. Loguéate
2. Ve a:

`http://localhost:8080/logout`

3. Debes acabar en:

`http://localhost:8080/`

Errores típicos

- “**Invalid logout_uri**”: la URL no está en “Allowed sign-out URLs” en Cognito.
 - Confundir `issuer-uri` con `domain`:
 - `issuer-uri`: `https://cognito-idp.../poolId`
 - `domain`: `https://xxxx.auth.region.amazoncognito.com`
-

Si me dices tu **region** y tu **domain prefix** (el `xxxx.auth...`), te escribo el `app.cognito.domain` exacto y te digo qué URLs exactas debes registrar en Cognito (local + futura producción).

You said:

se puede tener un fichero config default y luego otros por perfil. Por ejemplo si ejecuto perfil aws que se cargue el default + aws

ChatGPT said:

Sí, se puede (y es lo habitual).

En Spring Boot, `application.yml` (**default**) se carga siempre, y luego **se “superpone”** el fichero del perfil activo, por ejemplo `application-aws.yml`. Es decir:

- `application.yml` ☒ siempre
- `application-aws.yml` ☒ solo si `spring.profiles.active=aws`
- Las propiedades del perfil **sobrescriben** a las del default

Estructura típica

```
src/main/resources/  
  application.yml  
  application-local.yml  
  application-aws.yml
```

Ejemplo práctico

application.yml (default)

```
server:  
  port: 8080  
  
app:  
  name: library  
  cognito:  
    logout-redirect: http://localhost:8080/  
  
logging:  
  level:  
    org.springframework.security: INFO
```

application-aws.yml

```
app:  
  cognito:  
    logout-redirect: https://myapp.mydomain.com/  
  
spring:  
  security:  
    oauth2:  
      client:  
        registration:  
          cognito:  
            client-id: YOUR_CLIENT_ID  
            client-secret: YOUR_CLIENT_SECRET  
            scope: [openid, email, profile]  
        provider:  
          cognito:  
            issuer-uri: https://cognito-idp.eu-west-1.amazonaws.com/eu-west-1_XXXXXXX
```

Cuando ejecutas con perfil `aws`, tendrás:

- `server.port=8080` (del default)
 - `app.name=library` (del default)
 - `app.cognito.logout-redirect` sobrescrito por el de `aws`
 - toda la config de OAuth2 del fichero `aws`
-

Cómo activar perfil

En línea de comandos

```
java -jar app.jar --spring.profiles.active=aws
```

En IntelliJ (Run Config)

Añade:

```
--spring.profiles.active=aws
```

Consejo importante

No pongas secretos en `application.yml`.

Pon en default solo lo común, y en `application-aws.yml` usa variables:

```
spring:
  security:
    oauth2:
      client:
        registration:
          cognito:
            client-id: ${COGNITO_CLIENT_ID}
            client-secret: ${COGNITO_CLIENT_SECRET}
```

Si quieres, te dejo una plantilla completa `application.yml + application-local.yml + application-aws.yml` ya adaptada a tu login + logout con Cognito.

You said:

```
@Bean SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception { http .csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> auth
.requestMatchers("/", "/public/**", "/error").permitAll()
.requestMatchers("/admin/**").hasRole("LIBRARIAN")
.anyRequest().authenticated() ) .oauth2Login(oauth2 -> {}).logout(logout ->
logout .logoutUrl("/logout")
.logoutSuccessHandler(cognitoLogoutSuccessHandler()))
.invalidateHttpSession(true) .clearAuthentication(true)
.deleteCookies("JSESSIONID") ); return http.build(); } Description: Method
securityFilterChain in com.upm.library.security.AwsSecurityConfig required a
bean of type
'org.springframework.security.oauth2.client.registration.ClientRegistrationReposi
tory' that could not be found. Action: Consider defining a bean of type
'org.springframework.security.oauth2.client.registration.ClientRegistrationReposi
tory' in your configuration.
```

ChatGPT said:

Ese error significa que **Spring no está creando el "OAuth2 Client"** (y por eso no existe el bean `ClientRegistrationRepository`). Casi siempre pasa por una de estas razones:

1. **Falta la dependencia** `spring-boot-starter-oauth2-client`
2. Estás ejecutando con un perfil donde **no se carga** la config `spring.security.oauth2.client...`
3. Hay un typo en la ruta de configuración (`spring.security.oauth2.client`)

Como tú estás usando `.oauth2Login(...)`, **la causa #1 es la más frecuente.**

1) Asegúrate de tener la dependencia correcta

Gradle

```
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-security")
}
```

```
    implementation("org.springframework.boot:spring-boot-starter-oauth2-client")
}
```

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Sin esa dependencia, Spring no auto-configura `ClientRegistrationRepository`.

2) Verifica que la config de Cognito se está cargando en el perfil `aws`

Si tienes `application.yml` + `application-aws.yml`, la parte OAuth2 debe estar en el fichero que realmente se carga al ejecutar.

Ejemplo mínimo **en** `application-aws.yml`:

```
spring:
  security:
    oauth2:
      client:
        registration:
          cognito:
            client-id: ${COGNITO_CLIENT_ID}
            client-secret: ${COGNITO_CLIENT_SECRET}
            scope: [openid, email, profile]
            authorization-grant-type: authorization_code
            redirect-uri: "${baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          cognito:
            issuer-uri: https://cognito-idp.eu-west-1.amazonaws.com/eu-west-1_XXXXXXX
```

Y ejecuta con:

```
--spring.profiles.active=aws
```

Si ejecutas con `local` y tu config `oauth2` está solo en `aws`, entonces `ClientRegistrationRepository` no se crea (pero en `local` tampoco deberías tener `.oauth2Login()` activo).

3) Tu clase `AwsSecurityConfig` debe ser solo para perfil `aws`

Asegúrate de que tu config con `oauth2Login` está anotada así:

```
@Configuration
@Profile("aws")
public class AwsSecurityConfig { ... }
```

Y tu config "sin seguridad" en `local`:

```
@Configuration
@Profile("local")
public class LocalSecurityConfig {

    @Bean
    SecurityFilterChain localChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(a -> a.anyRequest().permitAll());
        return http.build();
    }
}
```

4) Si quieres que compile incluso sin `OAuth2` config (fallback)

No es lo ideal, pero si te bloquea, puedes hacer que `AwsSecurityConfig` no se cargue si no existe `ClientRegistrationRepository`:

```
@Configuration
@Profile("aws")
```



```
@ConditionalOnBean(ClientRegistrationRepository.class)
public class AwsSecurityConfig { ... }
```

Pero repito: lo correcto es **dependencia + properties**.

Resumen (lo que tienes que hacer ya)

- ✓ Añade `spring-boot-starter-oauth2-client`
 - ✓ Asegura que `spring.security.oauth2.client.registration...` está en el perfil activo (`aws`)
 - ✓ Asegura `@Profile("aws")` en esa config
-

Si me pegas tu `build.gradle` (dependencias) y el `application-aws.yml` (solo la parte de `spring.security.oauth2...`), te digo exactamente cuál de los 3 puntos es el culpable en tu proyecto.

You said:

teniendo en cuenta todo los requisitos y la estructura hasta ahora. Generame los datos para cargarlos a h2 y probar los endpoints

ChatGPT said:

Aquí tienes un **dataset completo** (en inglés, acorde a tu estructura actual) para cargar en **H2** y poder probar los endpoints del MVP.

Te dejo:

1. Config para H2 en perfil `local`
2. `data.sql` con datos coherentes (books, copies, users, loans, reservations, penalties)
3. 6 llamadas `curl` para probar endpoints (sin seguridad en `local`)

Asumo que en `local` **tienes security desactivada** (`permitAll`). Así puedes probar sin login.

1) application-local.yml (H2 + consola)

Crea src/main/resources/application-local.yml:

```
spring:
  datasource:
    url: jdbc:h2:mem:librarydb;MODE=PostgreSQL;DB_CLOSE_DELAY=-
1;DB_CLOSE_ON_EXIT=FALSE
    driver-class-name: org.h2.Driver
    username: sa
    password:

  jpa:
    hibernate:
      ddl-auto: create-drop
    properties:
      hibernate:
        format_sql: true

  h2:
    console:
      enabled: true
      path: /h2-console

logging:
  level:
    org.hibernate.SQL: debug
```

Y añade dependencia H2 si no la tienes:

Gradle

```
runtimeOnly("com.h2database:h2")
```

2) data.sql (para cargar datos en H2)

Crea src/main/resources/data.sql:

```

-- =====
-- USERS
-- =====

INSERT INTO users (id, external_id, role) VALUES
  (1, 'local-user-001', 'USER'),
  (2, 'local-librarian-001', 'LIBRARIAN');

-- =====
-- BOOKS
-- =====

INSERT INTO books (id, title, author, subject) VALUES
  (1, 'Clean Code', 'Robert C. Martin', 'Software Engineering'),
  (2, 'Domain-Driven Design', 'Eric Evans', 'Software Architecture'),
  (3, 'The Pragmatic Programmer', 'Andrew Hunt; David Thomas',
'Programming'),
  (4, 'Introduction to Algorithms', 'Cormen; Leiserson; Rivest; Stein',
'Algorithms'),
  (5, 'Design Patterns', 'Gamma; Helm; Johnson; Vlissides', 'Object-
Oriented Design');

-- =====
-- COPIES
-- status: AVAILABLE, RESERVED, LOANED
-- reference_only: true/false
-- =====

INSERT INTO copies (id, book_id, status, reference_only, location) VALUES
  (1, 1, 'AVAILABLE', FALSE, 'A1-01'),
  (2, 1, 'LOANED', FALSE, 'A1-02'),
  (3, 2, 'AVAILABLE', FALSE, 'A2-01'),
  (4, 2, 'RESERVED', FALSE, 'A2-02'),
  (5, 3, 'AVAILABLE', FALSE, 'A3-01'),
  (6, 3, 'AVAILABLE', TRUE, 'REF-01'),
  (7, 4, 'LOANED', FALSE, 'A4-01'),
  (8, 5, 'AVAILABLE', FALSE, 'A5-01');

-- =====
-- LOANS
-- closed: true/false
-- (Copy 2 and Copy 7 are LOANED)
-- =====

INSERT INTO loans (id, user_id, copy_id, start_date, due_date, renewals,
closed) VALUES
  (1, 1, 2, DATE '2026-01-10', DATE '2026-01-24', 0, FALSE),
  (2, 1, 7, DATE '2026-01-01', DATE '2026-01-15', 1, FALSE);

-- =====

```

```

-- RESERVATIONS
-- status: ACTIVE, CANCELED, FULFILLED, EXPIRED
-- (Copy 4 is RESERVED)
-- =====
INSERT INTO reservations (id, user_id, copy_id, status, created_at)
VALUES
  (1, 1, 4, 'ACTIVE', TIMESTAMP '2026-01-20 10:00:00'),
  (2, 1, 3, 'CANCELED', TIMESTAMP '2026-01-18 09:00:00');

-- =====
-- PENALTIES
-- active: true/false
-- =====
INSERT INTO penalties (id, user_id, start_date, end_date, active, reason)
VALUES
  (1, 1, DATE '2026-01-05', DATE '2026-01-12', FALSE, 'Old penalty
(expired)'),
  (2, 1, DATE '2026-01-16', DATE '2026-01-23', TRUE, 'Late return');

```

Nota importante sobre fechas

He puesto fechas alrededor de enero 2026 para que:

- haya préstamos “abiertos”
- haya una sanción activa (para probar reglas)

Si hoy en tu entorno la fecha no coincide y quieres evitar bloqueos por sanción, dime y lo adapto.

3) Probar endpoints (perfil local, sin auth)

Arranca con perfil local:

```
./gradlew bootRun --args="--spring.profiles.active=local"
```

A) Buscar catálogo

```
curl "http://localhost:8080/api/catalog"
curl "http://localhost:8080/api/catalog?query=design"
```

B) Ver detalle de un copy

```
curl "http://localhost:8080/api/copies/1"
```

C) Crear reserva (copy 1 está AVAILABLE)

```
curl -X POST "http://localhost:8080/api/reservations" \
  -H "Content-Type: application/json" \
  -d '{"copyId": 1}'
```

Si tu `ReservationService` usa el usuario por JWT `subject`, en `local` necesitas una estrategia.

Si en `local` permitAll pero tu service sigue esperando `externalUserId` desde JWT, te fallará.

Solución rápida: en `local` hardcodear el `subject`, o añadir un header tipo `X-User-Sub` y leerlo.

Si me dices cómo lo estás resolviendo ahora en `local`, te ajusto el dataset y los curls.

D) Cancelar reserva (la reserva 1 existe)

```
curl -X DELETE "http://localhost:8080/api/reservations/1"
```

E) Admin: crear préstamo (bibliotecario)

```
curl -X POST "http://localhost:8080/api/admin/loans" \
  -H "Content-Type: application/json" \
  -d '{"userId": 1, "copyId": 3}'
```

F) Admin: registrar devolución (copy 2 tiene préstamo abierto)

```
curl -X POST "http://localhost:8080/api/admin/returns" \
  -H "Content-Type: application/json" \
  -d '{"copyId": 2}'
```

Punto clave: “local sin seguridad” pero tus controllers usan `@AuthenticationPrincipal`

En tu código anterior, `ReservationController` y `MyAccountController` sacaban el `subject` del JWT (`Authentication auth`).

Si en `local` no hay login, `auth` será `null` y petará.

Tienes 2 opciones simples para `local`:

Opción 1 (recomendada): en `local` simular usuario con header

- En controller, si perfil local, lee `X-User-Sub`

- En aws, usa JWT/OIDC normal

Opción 2: crear un “LocalAuthenticationFilter” en perfil local

que inyecte un usuario fijo (local-user-001).

Si me dices cuál prefieres, te doy el código exacto (en inglés) para que tus endpoints funcionen en local con este dataset sin tocar lógica de negocio.

¿Tus controllers ahora mismo siguen leyendo el subject desde Authentication (JWT/OIDC), o ya los adaptaste para local?

You said:

Error creating bean with name 'dataSourceScriptDatabaseInitializer' defined in class path resource [org/springframework/boot/jdbc/autoconfigure/DataSourceInitializationAutoConfiguration.class]: Failed to execute SQL script statement #1 of file [C:\Users\Usuario\tfm\Library\target\classes\data.sql]: INSERT INTO users (id, external_id, role) VALUES (1, 'local-user-001', 'USER'), (2, 'local-librarian-001', 'LIBRARIAN')

ChatGPT said:

Thought for 12s

Ese error casi seguro no es por el SQL en sí, sino por **el orden de inicialización**:

Spring está intentando ejecutar data.sql **antes** de que Hibernate haya creado las tablas (ddl-auto: create-drop). Resultado: **la tabla users aún no existe**, y falla en la primera sentencia.

Esto es súper típico con Spring Boot + JPA + data.sql.

Arreglo (recomendado): deferir la carga de data.sql hasta después de JPA

En tu application-local.yml, añade:

```
spring:
  jpa:
    defer-datasource-initialization: true
```

Te quedaría así (mínimo):

```
spring:
  datasource:
    url: jdbc:h2:mem:librarydb;MODE=PostgreSQL;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
    driver-class-name: org.h2.Driver
    username: sa
    password:

  jpa:
    hibernate:
      ddl-auto: create-drop
      defer-datasource-initialization: true

  h2:
    console:
      enabled: true
      path: /h2-console
```

Reinicia la app y debería cargar `data.sql` sin petar.

Si aun así falla: probable “USERS” como palabra reservada/tabla del sistema

En algunas combinaciones, `users` puede dar guerra. Solución sencilla: cambia el nombre de la tabla en tu entidad `User`:

```
@Entity
@Table(name = "app_users")
public class User { ... }
```

Y en `data.sql` cambias `users` por `app_users`.

Qué haría yo para que no haya más sorpresas

1. Añadir `spring.jpa.defer-datasource-initialization=true` ✓
2. Si usas H2, renombrar tabla a `app_users` ✓ (muy robusto)

Si me pegas la línea completa del error (la parte que dice algo como *Table "USERS" not found* o *Syntax error*), te digo con 100% certeza si es **orden de inicialización** o **nombre de tabla**.